

NODKAPP

BLOC 2

Concevoir et développer des applications logicielles

Plateforme multi-tenant de bureau virtuel 2D

TITRE VISÉ Expert en développement logiciel	CODE RNCP 39583
CANDIDAT	DATE

SOMMAIRE

- 1 Protocole de déploiement continu
- 2 Protocole d'intégration continue
- 3 Architecture & prototype
- 4 Harnais de tests unitaires
- 5 Sécurité — couverture OWASP Top 10
- 6 Accessibilité
- 7 Historique et gestion des versions
- 8 Cahier de recettes
- 9 Plan de correction des bogues
- 10 Manuel de déploiement
- 11 Manuel d'utilisation
- 12 Manuel de mise à jour

1

Protocole de déploiement continu

Compétence C2.1.1

Livrable RNCP **C2.1.1** — Mettre en œuvre des environnements de déploiement et de test en y intégrant les outils de suivi de performance et de qualité.

Livrables attendus : *le protocole de déploiement continu + les critères de qualité et de performance.*

1. Vue d'ensemble

Le déploiement continu (CD) prend le relais de l'intégration continue : une fois le code fusionné et validé, il est automatiquement empaqueté en images Docker, publié, puis déployé sur le serveur, sans intervention manuelle.

```
CI verte (PR fusionnée)
  |
  ▼
push sur dev / main → CD (.github/workflows/cd.yml)
  |
  ├── 1. build images Docker (api + web)
  ├── 2. push sur GHCR (tags mutable + immuable)
  ├── 3. SSH vers le VPS OVH
  ├── 4. docker compose pull + up -d
  ├── 5. migrations Drizzle (au démarrage du conteneur api)
  └── 6. healthcheck HTTP
  ▼
Environnement en ligne
dev → https://dev.nookapp.eu    (staging)
main → https://nookapp.eu       (production)
```

2. Environnement de développement (détail)

COMPOSANT	CHOIX	RÔLE
Runtime	Node.js ≥ 22 (<code>engines</code>)	Exécution JS/TS côté serveur et build.
Gestionnaire de paquets	pnpm 10.18.2 (workspaces)	Monorepo : <code>apps/*</code> + <code>packages/*</code> .
Éditeur	VS Code	+ ESLint / Prettier / Volar (Vue).
Compilateur / transpileur	TypeScript 5.6 (<code>tsc</code>)	Typage strict, cible ES2022.
Build API	<code>nest build</code> (NestJS CLI)	Compile <code>apps/api</code> → <code>dist/</code> .
Build Web	<code>nuxt build</code>	Compile <code>apps/web</code> → <code>.output/</code> (serveur Nitro).
Serveur d'application (API)	NestJS 10 (HTTP + Socket.IO, port 4000 ; Hocuspocus port 4234)	Backend REST + temps réel.
Serveur d'application (Web)	Nitro (Nuxt 3, port 4001)	SSR + SPA.
Base de données	PostgreSQL 17	Données applicatives.
ORM / migrations	Drizzle + drizzle-kit	Schéma typé, migrations SQL versionnées.
Gestion de sources	Git + GitHub	Dépôt, PR, Actions.
Qualité de code	ESLint 9, Prettier 3	Lint + formatage.
Hooks Git	Husky + commitlint + lint-staged	Barrière qualité pré-commit.
Services locaux	Docker Compose (<code>docker-compose.yml</code>)	Postgres 17, Mailpit (emails), LiveKit (voix).

Démarrage local : `pnpm install` puis `pnpm dev` (lance en parallèle les packages en watch, l'API et le Web).

3. Environnements de déploiement

Le projet s'appuie sur **quatre environnements isolés**, chacun avec son propre cycle de vie et ses propres données. Cette séparation garantit qu'une expérimentation en développement ou une exécution de tests ne peut pas altérer les données de production.

ENVIRONNEMENT	LOCALISATION	BRANCHE	BASE DE DONNÉES	ACCÈS
Développement	Poste local	branche de ticket	Postgres 17 conteneurisé (<code>docker-compose.yml</code>)	<code>http://localhost:4001</code>
Test	Runner GitHub Actions (Ubuntu)	toutes	éphémère, recrée à chaque exécution	interne au run
Staging	VPS OVH — <code>/opt/nookapp-staging/</code>	<code>dev</code>	volume <code>nookapp-staging</code>	<code>https://dev.nookapp.eu</code>
Production	VPS OVH — <code>/opt/nookapp-prod/</code>	<code>main</code>	volume <code>nookapp-prod</code>	<code>https://nookapp.eu</code>

L'**environnement de test** est éphémère par conception : chaque exécution de CI part d'un runner neuf, réinstalle les dépendances depuis le lockfile (`--frozen-lockfile`) et détruit tout en fin de run. Aucun état ne survit d'une exécution à l'autre, donc aucun test ne peut réussir grâce à un résidu laissé par un run précédent.

Staging et production tournent sur le même VPS OVH (adresse IP non reproduite ici — elle figure dans les secrets GitHub `VPS_HOST` et dans la configuration DNS), isolés par projet Docker Compose distinct. Un reverse proxy **Traefik v3** (mutualisé sur l'hôte) route par nom de domaine via des labels de conteneurs et gère le HTTPS automatique (Let's Encrypt).

4. Le pipeline de déploiement (séquences)

Fichier : `.github/workflows/cd.yml`.

Séquence 1 — Build & publication des images (`build-and-push`)

- Déclenché sur `push` vers `main` ou `dev` (ou manuellement via `workflow_dispatch`).
- Construit deux images multi-étapes (builder → runtime `node:22-alpine`) :
- `apps/api/Dockerfile` → image **api** (NestJS, entypoint = migrations puis `node dist/main.js`).
- `apps/web/Dockerfile` → image **web** (Nuxt, sert `.output/server/index.mjs`).
- Publie sur **GHCR** (GitHub Container Registry) avec deux tags :

```
`` ghcr.io/<owner>/nookapp-api:<branche> # mutable (ex: main) ghcr.io/<owner>/nookapp-api:
<branche>-<sha12> # immuable (traçabilité) ``
```

Le tag immuable (`main-a1b2c3d4e5f6`) garantit qu'un déploiement pointe vers un artefact précis et rejouable. Cache de build via `type=gha`.

Séquence 2 — Déploiement (`deploy`)

Dépend de la séquence 1. Cible résolue selon la branche :

```
if [ "$REF" = "main" ]; then stack=nookapp-prod;    domain=nookapp.eu;
else                                     stack=nookapp-staging; domain=dev.nookapp.eu; fi
```

Étapes :

1. **Connexion SSH** au VPS (clé privée `secrets.VPS_SSH_KEY`, `ssh-keyscan` pour valider l'hôte).
2. **Mise à jour de la stack** : `` `bash cd /opt/${stack} \ && export IMAGE_TAG=${branche}-${sha} \ && docker compose -p ${stack} -f docker-compose.app.yml --env-file .env pull \ && docker compose -p ${stack} -f docker-compose.app.yml --env-file .env up -d --remove-orphans` ``
3. **Migrations de base de données** — exécutées automatiquement par `apps/api/docker-entrypoint.sh` au démarrage du conteneur api, **avant** de lancer le serveur : `` `sh node ./node_modules/@nookapp/db/dist/migrate.js # applique les migrations Drizzle exec node dist/main.js # démarre l'API` ``
4. **Healthcheck** (vérification post-déploiement) : `` `bash for i in 1..6; do sleep 10 code=$(curl -s -o /dev/null -w "%{http_code}" "https://${domain}/api/v1/health") [ "$code" = "200" ] && exit 0 done exit 1` `` L'endpoint `/api/v1/health` renvoie `{status,db}` après un `SELECT 1` : il valide que l'API répond et que la base est joignable. 6 tentatives, 10 s d'intervalle.

Progressivité et retour arrière

- Le déploiement est **progressif par environnement** : tout passe d'abord par `dev` (staging) avant d'atteindre `main` (production).
- `docker compose up -d` recrée les conteneurs applicatifs sans toucher aux volumes (`postgres-data`, `api-uploads`).
- **Rollback** : redéployer un tag immuable antérieur (`IMAGE_TAG=main-<ancien-sha>`) suffit à revenir à une version connue.

5. Composants identifiés (compilateur, serveur d'application, gestion de sources)

Conformément aux attendus du référentiel, le protocole met en évidence :

- **Compilateur / build** : `tsc` (typage), `nest build` (API), `nuxt build` (Web) — exécutés dans l'étape *builder* des Dockerfiles.
- **Serveur d'application** : NestJS (Node) pour l'API, Nitro (Node) pour le Web, PostgreSQL 17 pour les données, LiveKit pour la voix — orchestrés par `docker-compose.app.yml` derrière Traefik.
- **Outils de gestion de sources** : Git + GitHub (dépôt, PR, Actions), GHCR (registre d'images), volumes Docker (persistance).

6. Critères de qualité et de performance

Ces critères sont **automatiquement vérifiés** dans la chaîne CI/CD et répondent aux exigences du projet (application temps réel multi-tenant).

Critères de qualité

CRITÈRE	OUTIL / MÉCANISME	SEUIL / RÈGLE
Style et conventions	ESLint + Prettier (CI + pre-commit)	0 erreur pour fusionner.
Typage	<code>tsc strict (strict , noUncheckedIndexedAccess)</code>	0 erreur de type.
Tests unitaires	Jest (152 tests, 17 suites) + Vitest	100 % au vert.
Buildabilité	<code>pnpm build</code> (API + Web)	Build réussi obligatoire.
Format des commits	commitlint (Conventional Commits)	Rejet sinon.
Fraîcheur des dépendances	Dependabot (npm hebdo, Actions mensuel)	PR de mise à jour groupées.
Reproductibilité	<code>pnpm install --frozen-lockfile</code>	Dépendances figées.
Traçabilité des artefacts	Tag Docker immuable <code><branche>-<sha></code>	1 image = 1 commit.

Critères de performance

CRITÈRE	MÉCANISME	CIBLE
Disponibilité après déploiement	Healthcheck <code>/api/v1/health</code> (6×10 s)	HTTP 200 avant de valider le déploiement.
Santé base de données	<code>SELECT 1</code> dans le healthcheck	<code>db: true</code> .
Démarrage conteneur	<code>tini</code> (PID 1), image <code>alpine</code> légère	Boot rapide, gestion propre des signaux.
Persistance base	Healthcheck Compose <code>pg_isready</code>	api démarre après <code>postgres</code> sain (<code>depends_on</code>).
Latence temps réel	Socket.IO + LiveKit (ports dédiés)	Interaction fluide (chat, positions, voix).
Sauvegarde	Conteneur <code>db-backup</code> (pg_dump planifié + rétention)	Point de restauration régulier.

Outils de suivi mobilisés, par environnement

Le référentiel demande d'intégrer les outils de suivi de performance et de qualité aux environnements. Le tableau récapitule ceux effectivement en place et l'environnement où chacun s'exécute.

OUTIL	ENVIRONNEMENT	CE QU'IL SUIT	DÉCLENCHEMENT
ESLint + Prettier	Dev, Test	Conformité de style	Pre-commit (lint-staged) + job CI <code>lint</code>
<code>tsc --noEmit</code>	Dev, Test	Erreurs de typage	Job CI <code>typecheck</code>
commitlint	Dev	Format des messages de commit	Hook <code>commit-msg</code>
Jest + <code>--coverage</code>	Dev, Test	Régressions + taux de couverture	Job CI <code>test</code> , seuil bloquant
Artefact <code>coverage-api</code>	Test	Rapport HTML archivé par exécution	Job CI <code>test</code> (<code>upload-artifact</code>)
GitHub Actions	Test	Durée et statut de chaque exécution	Chaque push et PR
Swagger (<code>/api/docs</code>)	Tous	Contrat d'API généré depuis les DTO Zod	Permanent
Healthcheck <code>/api/v1/health</code>	Staging, Prod	Disponibilité API + joignabilité base (<code>SELECT 1</code>)	Post-déploiement, 6 tentatives bloquantes
Healthcheck Compose <code>pg_isready</code>	Staging, Prod	Santé du conteneur PostgreSQL	Continu (<code>depends_on</code>)
Journaux Docker	Staging, Prod	Sortie des conteneurs (<code>docker compose logs</code>)	À la demande
Sentry (API)	Staging, Prod	Exceptions serveur (≥ 500), traces de performance	Continu, dès que <code>SENTRY_DSN</code> est renseigné
Sentry (Web)	Staging, Prod	Exceptions JavaScript côté navigateur	Continu, dès que <code>SENTRY_DSN</code> est renseigné
<code>db-backup</code> (pg_dump planifié)	Staging, Prod	Existence d'un point de restauration	Planifié

La chaîne est donc outillée **de bout en bout du cycle** : le poste de développement (hooks), l'environnement de test (5 jobs CI + seuil de couverture + artefact), et les environnements déployés (healthchecks bloquants, sauvegardes). Un défaut de qualité est arrêté au plus tôt — au commit s'il est stylistique, en CI s'il est fonctionnel, au healthcheck s'il est opérationnel.

Capture des exceptions en production (Sentry)

Deux anomalies du registre (BUG-08, BUG-09) ont été découvertes par des retours d'usage plutôt que par un outil, et une troisième (BUG-10) en recette alors qu'elle se manifestait silencieusement en production. Ce constat a motivé l'ajout d'une **capture centralisée des exceptions**, sur les deux couches :

COUCHE	SDK	MISE EN ŒUVRE
API (Node)	<code>@sentry/node</code>	Initialisé en tout premier dans <code>src/instrument.ts</code> , importé en tête de <code>main.ts</code> pour que l'instrumentation précède le chargement des modules Node.
Web (navigateur)	<code>@sentry/nuxt</code>	Module Nuxt + <code>sentry.client.config.ts</code> .

Trois choix de conception méritent d'être explicités :

- **Filtrage du bruit.** Un filtre d'exceptions global (`common/sentry-exception.filter.ts`) ne remonte que les erreurs ≥ 500 . Les 4xx sont des erreurs d'appelant attendues (validation Zod, permission refusée) : les remonter noierait les vraies anomalies serveur. Le comportement est verrouillé par cinq tests unitaires.
- **Inerte sans DSN.** Sans `SENTRY_DSN`, le SDK n'est pas initialisé du tout : aucun appel réseau, aucun compte requis pour développer, et un build de CI qui n'envoie rien. La configuration est donc optionnelle par construction.
- **Traçabilité de version.** La CD injecte `SENTRY_RELEASE` avec **la même valeur que le tag d'image immuable** (`<branche>-<sha12>`). Une exception remontée est donc reliée au commit exact qui l'a produite, et à l'artefact Docker déployé.

Les environnements sont distingués par `SENTRY_ENVIRONMENT` (`staging` /

Condition d'activation. L'intégration est codée, testée et câblée jusque dans le déploiement, mais elle ne devient effective qu'une fois `SENTRY_DSN` renseigné dans le fichier `.env` de la stack concernée sur le serveur. Tant que la variable est vide, le SDK n'est pas initialisé et **aucune exception n'est remontée** : c'est le comportement voulu en développement, mais cela signifie que la mise en service en production suppose cette étape de configuration.

Limite restante. Sentry couvre les **exceptions**, pas les **métriques système** (CPU, mémoire, latence, saturation disque) ni la **journalisation centralisée**. Ce lot reste cadré pour le **Bloc 4 (C4.1.2)** : Prometheus + Loki + Promtail + Grafana dans un `docker-compose.monitoring.yml` séparé, avec alertes vers un webhook Discord. Le rejeu de session Sentry est par ailleurs désactivé volontairement : le monde 2D étant un canvas, il n'aurait rien à restituer et consommerait le quota pour un résultat vide.

Compétence C2.1.2

Livrable RNCP **C2.1.2** — Configurer le système d'intégration continue dans le cycle de développement du logiciel en fusionnant les codes sources et en testant régulièrement les blocs de code afin d'assurer un développement efficient qui réduit les risques de régression.

1. Objectif

L'intégration continue (CI) vérifie automatiquement, à chaque proposition de fusion de code, que le dépôt reste dans un état sain : code formaté, typé, testé et compilable. Elle réduit les risques de régression en détectant les défauts avant qu'ils n'atteignent la branche d'intégration.

2. Outil et emplacement

- **Plateforme** : GitHub Actions.
- **Fichier** : `.github/workflows/ci.yml`.
- **Gestionnaire de sources** : Git (dépôt GitHub), monorepo npm.

3. Déclenchement (quand la CI s'exécute)

```
on:
  pull_request:
    branches: [dev, main]
  push:
    branches: [dev, main]
concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

- **Sur chaque Pull Request** vers `dev` ou `main` : c'est la barrière de fusion.
- **Sur chaque push** direct vers `dev` ou `main`.
- **Concurrence** : un nouveau push sur une branche annule le run précédent de cette branche (`cancel-in-progress: true`) pour ne pas gaspiller de ressources.

Ce déclenchement matérialise le principe d'intégration **continue** : le code de chaque contributeur est fusionné et testé fréquemment, par petits incréments (une PR par ticket, cf. workflow Git du projet).

4. Séquences d'intégration (les jobs)

Le pipeline est découpé en 4 jobs parallèles + 1 job de synthèse. Chacun réinstalle les dépendances de façon reproductible (`pnpm install`)

SÉQUENCE	JOB	COMMANDE	RÔLE
1	lint	pnpm lint	ESLint : qualité et style du code (TS + Vue).
2	typecheck	build packages puis pnpm typecheck	tsc : vérifie le typage strict sur tout le monorepo.
3	test	build packages puis pnpm test	Jest (API) + Vitest (Web) : tests unitaires.
4	build	pnpm build	Compile l'ensemble (API NestJS + Web Nuxt + packages).
5	ci-success	agrégation	Échoue si l'un des 4 jobs échoue. Point de contrôle unique.

Les jobs 2, 3 et 4 commencent par **construire les packages partagés** (`packages/protocol`, `packages/db`) car l'API et le Web en dépendent :

```
pnpm -r --filter "./packages/*" run build
```

Job de synthèse `ci-success`

```
ci-success:
  needs: [lint, typecheck, test, build]
  if: always()
  steps:
    - run: |
        if [ "${{ needs.lint.result }}" ≠ "success" ] || \
          [ "${{ needs.typecheck.result }}" ≠ "success" ] || \
          [ "${{ needs.test.result }}" ≠ "success" ] || \
          [ "${{ needs.build.result }}" ≠ "success" ]; then
          exit 1
        fi
```

Ce job unique est configuré comme **check requis** côté GitHub (branch protection) : une PR ne peut être fusionnée que s'il est vert. On n'a ainsi qu'un seul statut à exiger, quel que soit le nombre de jobs.



Figure 1.1 — Une exécution de CI complète. Les quatre jobs en parallèle et le job de synthèse `ci-success`, avec les durées réelles.



Figure 1.2 — Protection de branche. `ci-success` déclaré comme check requis sur `dev` et `main` — la preuve que le gate est réellement appliqué et pas seulement défini dans le workflow.

5. Enchaînement dans le cycle de développement



Deux barrières complémentaires :

1. **En local (Husky)** — avant même le commit : `.husky/pre-commit` lance `lint-staged` (ESLint `--fix` + Prettier sur les fichiers modifiés) et `.husky/commit-msg` lance `commitlint`. Feedback immédiat, moins

de bruit dans la CI.

2. En CI (GitHub Actions) — sur l'environnement neutre, la totalité du dépôt est revalidée avant fusion.

6. En quoi cela réduit les régressions

- **Tests systématiques** : les 152 tests unitaires (services API, guards, mailer, permissions) tournent à chaque PR — voir « Harnais de tests unitaires ».
- **Typage strict** : `tsc --noEmit` avec `strict: true` + `noUncheckedIndexedAccess` empêche une classe entière de bugs de fusionner.
- **Build réel** : le job `build` garantit que la version fusionnée compile vraiment (API + Web), pas seulement qu'elle « type ».
- **Lockfile figé** : `--frozen-lockfile` garantit des dépendances identiques entre la machine du développeur et la CI (pas de dérive silencieuse).
- **Isolation** : chaque job repart d'un runner propre `ubuntu-latest`.

7. Lien avec le déploiement

La CI ne déploie pas : elle valide. Une fois la PR fusionnée sur `dev` ou

(construction des images Docker, publication, déploiement VPS). Voir « Protocole de déploiement continu ».

Compétence C2.2.1

Livrable RNCP **C2.2.1** — Concevoir un prototype de l'application logicielle en tenant compte des spécificités ergonomiques et des équipements ciblés afin de répondre aux fonctionnalités attendues et aux exigences de sécurité.

Livrables : *architecture structurée permettant la maintenabilité, présentation d'un prototype, frameworks et paradigmes de développement.*

1. Présentation du prototype

NookApp est une plateforme multi-tenant de « bureau virtuel » (type Discord / Gather) : chaque utilisateur crée son **Nook** (serveur), invite des membres, discute en temps réel et évolue dans un monde 2D pixel-art où la voix se déclenche par proximité.

Le prototype livré (V1) est **fonctionnel et déployé** en continu sur <https://dev.nookapp.eu>. Il couvre un ensemble cohérent de fonctionnalités principales et les user stories associées :

USER STORY	FONCTIONNALITÉ LIVRÉE
En tant que visiteur, je m'inscris et je vérifie mon email.	Auth email+password, vérification, reset (Better Auth).
En tant qu'utilisateur, je crée mon Nook.	CRUD serveurs, slug unique, map par défaut.
En tant qu'admin, j'invite des membres.	Liens d'invitation, rejoindre via code.
En tant qu'admin, je modère.	Rôles/permissions, kick, ban, débannissement.
En tant que membre, je discute.	Salons texte, messages temps réel, édition/suppression, DM.
En tant que membre, j'évolue dans le monde.	Monde Phaser, déplacement, multijoueur, voix de proximité.
En tant qu'admin, j'édite la carte.	Éditeur collaboratif (sols/pièces/décor) via CRDT.
En tant qu'utilisateur, je gère mes données.	Contrôles RGPD (export, suppression).

Composants d'interface (présents et fonctionnels)

Fenêtres et modales (création de Nook, invitation, réglages, édition de salon/catégorie), menus (sidebar double avec dock, sélecteur de serveur), boutons d'action, formulaires labellisés (auth), panneau de chat détachable,

palette d'éditeur de map, HUD du monde. Interface **bilingue FR/EN** et **multi-thèmes** (clair / sombre / pixel-quest).



Figure 2.1 — Le monde 2D en session. Deux joueurs, étiquettes de nom projetées en DOM au-dessus du canvas, sidebar des salons à gauche, panneau de chat à droite, dock en bas.



Figure 2.2 — L'éditeur de map. Palette des trois phases (sols / pièces / décor), grille 32×32, aperçu de la pièce en cours de tracé.



Figure 2.3 — Chat temps réel et liste des salons. Salons thématiques avec icônes et état de lecture, catégories repliables, message en cours de rédaction.



Figure 2.4 — Réglages des rôles. Création d'un rôle, cases de permissions, hiérarchie par position — la surface qui pilote `roles.service.ts`.



Figure 2.6 — Les trois thèmes. Clair, sombre et pixel-quest sur le même écran, illustrant les tokens de couleur et les contrastes contrôlés au chapitre « Accessibilité ».

Équipements ciblés (spécificités ergonomiques)

Cible **web desktop** (l'expérience monde 2D + voix + partage d'écran suppose un écran et un clavier). Rendu **pixel-perfect** (`image-rendering: pixelated`), d'où une règle d'architecture : **le texte est en DOM, jamais dans le canvas** Phaser (les libellés/badges sont projetés par-dessus le canvas). Accessibilité des interfaces autour du canvas traitée séparément — « Accessibilité ».

2. Frameworks et paradigmes de développement

COUCHE	FRAMEWORK / TECHNO	PARADIGME
Backend	NestJS 10	Modulaire, injection de dépendances , décorateurs, guards/pipes/intercepteurs.
ORM	Drizzle	Schéma-as-code typé, requêtes paramétrées.
Base	PostgreSQL 17	Relationnel + JSONB (état de la carte).
Auth	Better Auth	Email+password, sessions en base, vérification email.
Frontend	Nuxt 3 / Vue 3	Composition API , SSR/SPA par route, composables.
État	Pinia	Store réactif centralisé.
Moteur de jeu	Phaser 3	Boucle de rendu, scènes, systèmes.
Temps réel	Socket.IO	Événements <code>domaine:action</code> , rooms par serveur.
Collaboration	Y.js + Hocuspocus	CRDT (résolution de conflits sans verrou).
Voix / vidéo	LiveKit	WebRTC, salles par canal vocal.
Contrats	Zod	Schema-first : validation + inférence de types.

Paradigme transverse — « **Zod-first** » : un schéma Zod est défini une seule fois dans `packages/protocol`, le type TypeScript en est **inféré** (`z.infer`), et le même schéma valide les données **au bord** (contrôleur, gateway, fetch client). Un seul contrat, partagé client ↔ serveur, impossible à désynchroniser.

3. Patrons de conception mis en œuvre

Les patrons ci-dessous ne sont pas cités pour la forme : chacun renvoie à un fichier du dépôt et résout un problème concret rencontré pendant le développement.

PATRON	MISE EN ŒUVRE	PROBLÈME RÉSOLU
Fabrique	<code>mailer/mailer.factory.ts</code> — <code>makeDriverFactory()</code> construit le driver d'email d'après <code>MAIL_DRIVER</code>	Le service d'email diffère par environnement (Mailpit en local, Resend en production) sans que le code appelant ne le sache
Stratégie	Interface <code>MailerDriver</code> (<code>mailer.types.ts</code>) implémentée par <code>MailpitDriver</code> et <code>ResendDriver</code>	Deux façons d'envoyer un email, interchangeables derrière un contrat à une seule méthode <code>send()</code>
Injection de dépendances	Jetons <code>MAILER_DRIVER</code> et <code>DB</code> (Symbol) injectés par le conteneur NestJS	Les services ne construisent jamais leurs dépendances ; en test, le jeton est remplacé par un double sans toucher au service
Chaîne de responsabilité	<code>@UseGuards(AuthGuard, ServerScopeGuard)</code> puis <code>PermissionsGuard</code>	Authentifier, puis résoudre l'appartenance au Nook, puis vérifier la permission — chaque maillon peut interrompre la chaîne, aucun ne connaît les autres
Décorateur	<code>current-user.decorator.ts</code> , <code>current-member.decorator.ts</code> , <code>current-authz.decorator.ts</code>	Le contrôleur reçoit directement l'utilisateur, le membre et ses permissions en paramètre, sans relire la requête
Pipe / validation au bord	<code>common/zod.pipe.ts</code>	Aucune donnée non validée n'atteint un service ; l'erreur est normalisée en <code>{ code, message, details }</code>
DTO	Schémas Zod de <code>packages/protocol/src/dto/</code>	Un schéma distinct par opération (création, mise à jour, réponse) plutôt qu'un objet fourre-tout
Observateur	Passerelle Socket.IO côté serveur, stores Pinia côté client	Un changement d'état est diffusé aux abonnés sans que l'émetteur connaisse les destinataires

4. Principes SOLID

PRINCIPE	APPLICATION DANS LE CODE
Responsabilité unique	Contrôleurs fins / services gras : le contrôleur valide et délègue, le service porte la logique et est seul à toucher la base. Un module par domaine.
Ouvert / fermé	Ajouter un domaine, c'est ajouter un module NestJS et l'enregistrer — aucun fichier existant n'est modifié.
Substitution de Liskov	<code>MailpitDriver</code> et <code>ResendDriver</code> sont substituables sans condition : <code>mailer.service.ts</code> ne teste jamais lequel il détient. <code>mailer.module.spec.ts</code> vérifie les deux chemins.
Ségrégation des interfaces	<code>MailerDriver</code> n'expose qu'une méthode. Côté contrats, <code>createMessageSchema</code> et <code>updateMessageSchema</code> sont distincts au lieu d'un schéma unique aux champs optionnels.
Inversion des dépendances	Les services dépendent du jeton abstrait <code>DB</code> , jamais du client Drizzle concret. C'est ce qui rend les 152 tests unitaires exécutables sans base de données.

L'inversion des dépendances est le principe le plus structurant du projet : sans elle, tester `roles.service.ts` — 417 lignes de logique de permissions — supposerait une base PostgreSQL réelle, et le harnais ne tournerait ni en CI ni en boucle courte pendant le développement.

5. Architecture — vue d'ensemble (maintenabilité)

Monorepo **pnpm workspaces** : le code est organisé **par fonctionnalité** (feature-first), jamais par couche technique.

```
nookapp/  
├─ apps/  
│   ├─ api/      NestJS — 1 module par domaine  
│   └─ web/      Nuxt 3 — pages / composables / stores / world (Phaser)  
├─ packages/  
│   ├─ protocol/ Schémas Zod partagés (DTO REST, events socket, permissions)  
│   └─ db/        Schéma Drizzle + migrations + seed
```

Dépendances : `apps/api` et `apps/web` importent `@nookapp/protocol` et cohérence.

Backend NestJS — modules par domaine

Chaque domaine possède son module (service + contrôleur + guards + DTO), rien ne fuit hors du module sauf export explicite :

Un système de plugins par Nook a fait l'objet de travaux exploratoires (SDK, passerelle d'événements) qui ne sont **pas inclus dans la version livrée** : la fonctionnalité est reportée après la V1 et n'est donc décrite ni dans le cahier de recettes ni dans les manuels.

Convention respectée : **contrôleurs fins, services gras**. Le contrôleur valide et délègue ; le service porte la logique et est le seul à toucher au dépôt.

Multi-tenancy (sécurité par construction)

Chaque table possédée par un tenant porte un `serverId`. Une **chaîne de guards** protège chaque route :

```
@UseGuards(AuthGuard, ServerScopeGuard) + @RequirePermission(...)
|                                     |
session valide ?   membre du serveur ?   permission (bitfield) ?
                   injecte req.server/req.member
```

- `AuthGuard` — résout la session Better Auth ou rejette (401).
- `ServerScopeGuard` — vérifie l'appartenance et **injecte** `req.server` + `req.member` dans le contexte, empêchant une requête inter-tenant par construction.
- `PermissionsGuard` — vérifie les permissions granulaires (bitfield ; propriétaire = toutes permissions).

Rooms Socket.IO nommées `server:${serverId}`, salles LiveKit

réel et la voix.

Frontend Nuxt

- Pages** : `/` et `/legal/**` en pré-rendu, `/auth/**` en SSR, `/app/**` en SPA (`routeRules`).
- Composables** (≈ 45) : couche fine reliant stores et vues (`useSocket`, `useVoice`, `useMap`, `useChannels` ...). La logique métier vit dans les stores Pinia, pas dans les composants.
- Monde** : composants `components/world/` orchestrant Phaser ; sous-systèmes isolés (character, player, build, rooms, overlays). Layout de sprite-sheet centralisé dans `utils/cg-sheet.ts` (importé par la scène ET l'aperçu Vue pour éviter toute dérive).

Deux couches temps réel séparées (choix d'architecture)

- Socket.IO** — présence, positions (15 Hz), chat, état voix. Pas de sémantique CRDT nécessaire.
- Y.js / Hocuspocus** — édition **collaborative** de la map (plusieurs admins simultanés) ; résolution de conflits CRDT, persistance débouçée en JSONB.

Voix/vidéo entièrement dans **LiveKit** (chemin média séparé, tokens JWT courts scopés par salle).

6. Modèle de données (Drizzle)

24 fichiers de schéma, un par domaine, indexés sur chaque `serverId` :

- Auth** : `user`, `session`, `account`, `verification` (Better Auth).

- **Serveurs** : `server`, `member`, `server_invite`, `server_ban`.
- **Salons** : `channel`, `channel_category`, `message`.
- **Rôles** : `role`, `member_role` (n-n).
- **DM** : `conversation`, `direct_message`, `participant`.
- **Map** : `map` (Y.Doc sérialisé en JSONB).

7. Exigences de sécurité (satisfaites par le prototype)

Le prototype intègre la sécurité dès la conception : guards multi-tenant, validation Zod systématique aux frontières (anti-injection), ORM paramétré, Better Auth (mots de passe hachés, sessions en base), rate limiting, Helmet, CORS verrouillé. Le détail figure dans « Sécurité — couverture OWASP Top 10 ».

8. En quoi l'architecture est maintenable, sécurisée, extensible

- **Maintenable** — feature-first, contrat partagé unique, tests par service. La convention interne vise des unités courtes (< 50 lignes par fonction, < 200 lignes par fichier) ; elle est tenue sur les modules récents, mais quelques composants historiques la dépassent largement (`UserSettingsModal.vue`, `BuildPanel.vue`, `SideBar.vue`) et constituent la dette de refactorisation identifiée.
- **Sécurisée** — chaîne de guards, scoping par construction, validation au bord, secrets hors dépôt.
- **Extensible** — nouveaux domaines = nouveaux modules NestJS isolés ; nouveaux thèmes et nouvelles langues sans toucher au métier ; contrat Zod partagé qui propage tout changement d'API des deux côtés.

4 Harnais de tests unitaires

Compétence C2.2.2

Livrable RNCP **C2.2.2** — Développer un harnais de test unitaire en tenant compte des fonctionnalités demandées afin de prévenir les régressions et de s'assurer du bon fonctionnement du logiciel.

Livrable : *un jeu de tests unitaires couvrant une fonctionnalité demandée*. Critère : *les tests unitaires couvrent la majorité du code développé*.

1. Outillage

- **Framework (API)** : Jest 29 + `ts-jest`.
- **Config** : `apps/api/jest.config.js` (`testRegex: '.*\\.spec\\.ts$'`, environnement `node`, dossier `coverage/`).
- **Framework (Web)** : Vitest (`vitest run`).
- **Convention du projet** : chaque service possède un `*.spec.ts` (unitaire, dépendances mockées).
- **Exécution** : `pnpm test` (racine, tout le monorepo) ou `pnpm --filter @nookapp/api test`. Couverture : `pnpm --filter @nookapp/api test -- --coverage`.
- **Automatisation** : ces tests s'exécutent à chaque PR dans le job `test` de la CI (voir « Protocole d'intégration continue »).

2. Stratégie de test

Le harnais cible le **cœur métier et sécuritaire** de l'API : autorisation, permissions, modération, isolation multi-tenant, intégrité des messages, envoi d'emails. Ce sont les zones où une régression est la plus coûteuse (fuite de données inter-tenant, escalade de privilèges, message d'un autre auteur édité...).

Paradigme : tests unitaires isolés. Les dépendances (base Drizzle,

(une douzaine de secondes pour la suite complète) et déterministes.

Patron de mock de la base

Drizzle expose une API *chaînable* (`select().from().where().limit()`). Chaque maillon est mocké et le dernier renvoie la donnée simulée :

```
const chain = {
  from: jest.fn().mockReturnThis(),
  where: jest.fn().mockReturnThis(),
  limit: jest.fn().mockResolvedValue([{ id: 'c1' }]),
};
mockDb.select.mockReturnValue(chain);
```

Le service est instancié via le conteneur d'injection NestJS de test :

```
const module = await Test.createTestingModule({
  providers: [
    ServiceÀTester,
    { provide: DB, useValue: mockDb },
    { provide: RolesService, useValue: mockRoles },
  ],
}).compile();
```


3. Jeu de tests — 17 suites, 152 cas

SUITE	FONCTIONNALITÉ COUVERTE	CAS
<code>roles.service.spec.ts</code>	Rôles & permissions (bitfields, hiérarchie, escalade)	44
<code>dms.service.spec.ts</code>	Messages privés (conversations, participants)	19
<code>collaboration.service.spec.ts</code>	Édition collaborative (jetons, Y.Doc, anti-rebond)	15
<code>categories.service.spec.ts</code>	Catégories de salons	12
<code>maps.service.spec.ts</code>	Cartes (chargement, validation, repli)	9
<code>permissions.guard.spec.ts</code>	Contrôle de permission par route	8
<code>messages.service.spec.ts</code>	Messages (liste, création, édition, suppression)	7
<code>members.service.spec.ts</code>	Membres — kick / ban (modération)	6
<code>voice.service.spec.ts</code>	Jetons LiveKit (portée par salle)	6
<code>sentry-exception.filter.spec.ts</code>	Remontée des erreurs serveur, filtrage des 4xx	5
<code>servers.service.spec.ts</code>	Serveurs (liste, join via invite, delete)	4
<code>mailer.module.spec.ts</code>	Fabrique de driver (Mailpit/Resend)	4
<code>channels.service.spec.ts</code>	Salons (accès, permission, appartenance)	3
<code>users.service.spec.ts</code>	Compte (suppression RGPD + transfert propriété)	3
<code>mailer.service.spec.ts</code>	Envoi email + échappement HTML (anti-XSS)	3
<code>auth.guard.spec.ts</code>	Authentification (guard)	2
<code>health.controller.spec.ts</code>	Healthcheck (état base)	2
Total		152

Aucun test ne touche une base réelle : le client Drizzle est simulé, les SDK externes (LiveKit, Hocuspocus, Resend) sont interceptés. La suite complète s'exécute en **moins de 10 secondes**, ce qui la rend utilisable en boucle courte pendant le développement et non seulement en CI.

4. Détail d'une fonctionnalité demandée : la modération (ban / kick)

fonctionnalité sensible développée pendant le durcissement V1 :

kickMember

- ✓ refuse de se kick soi-même
- ✓ refuse un demandeur sans permission ManageMembers
- ✓ refuse de kick le propriétaire du serveur
- ✓ refuse de kick un membre de rang égal ou supérieur

banMember

- ✓ refuse le ban sans permission ManageMembers
- ✓ enregistre le ban et retire le membre quand autorisé

Chaque cas vérifie une **règle métier de sécurité** : pas d'auto-modération, pas d'escalade de privilèges, protection du propriétaire, hiérarchie des rôles respectée. Le dernier vérifie le chemin nominal (insertion du ban + suppression du membre).

Autres cas notables

- **Isolation multi-tenant** — `messages / channels` : `ForbiddenException` si le salon n'appartient pas au serveur, ou si l'utilisateur n'est pas membre.
- **Intégrité des messages** — `updateMessage` refuse d'éditer le message d'un autre auteur ; `deleteMessage` autorise un non-auteur **uniquement** avec la permission `ManageMessages`.
- **Anti-XSS mailer** — `sendVerificationEmail` avec un nom `<script>alert(1)</script>` : le test vérifie que la sortie HTML est échappée (`<script>`).
- **Configuration** — la fabrique de mailer lève une erreur explicite si `MAIL_DRIVER=resend` sans clé API, ou pour un driver inconnu.

5. Couverture mesurée

La couverture est mesurée par `jest --coverage` et imposée en CI par un seuil minimal (`coverageThreshold` dans `apps/api/jest.config.js`) : tout passage sous ce seuil casse le job `test` et bloque la fusion.

```
$ pnpm --filter @nookapp/api exec jest --coverage
```

```
Test Suites: 17 passed, 17 total
Tests:      152 passed, 152 total
Time:       12.3 s
```

```
===== Coverage summary =====
Statements : 48.66% ( 691/1420 )
Branches   : 40.45% ( 250/618 )
Functions  : 44.40% ( 123/277 )
Lines      : 47.83% ( 585/1223 )
=====
```

Le dénominateur exclut le code purement déclaratif sans logique testable (modules de câblage DI, `main.ts`, décorateurs de paramètres) — ces fichiers n'ont pas de branche à couvrir et fausseraient la mesure à la baisse.

Couverture par service métier. Le chiffre global est tiré vers le bas par la couche contrôleurs (voir limites ci-dessous). Sur les **services**, où réside la logique, la couverture est la suivante :

SERVICE	STMTS	BRANCHES	COMMENTAIRE
<code>collaboration.service.ts</code>	100 %	100 %	Jetons, hydratation Y.Doc, debounce
<code>voice.service.ts</code>	100 %	85.7 %	Portée des jetons LiveKit
<code>categories.service.ts</code>	100 %	86.2 %	
<code>roles.service.ts</code>	99.4 %	95.1 %	Cœur des permissions
<code>maps.service.ts</code>	97.1 %	80.8 %	Repli sur carte par défaut
<code>dms.service.ts</code>	95.5 %	84.8 %	Contrôle de participation
<code>messages.service.ts</code>	80.4 %	88.2 %	
<code>permissions.guard.ts</code>	100 %	86.7 %	Garde de permission par route
<code>channels.service.ts</code>	59.1 %	28.1 %	À renforcer
<code>members.service.ts</code>	45.2 %	35.1 %	À renforcer
<code>users.service.ts</code>	42.2 %	31.3 %	À renforcer
<code>servers.service.ts</code>	38.5 %	26.1 %	À renforcer

L'effort a été porté en priorité sur `roles.service.ts` : 414 lignes de logique de permissions (bitfields, hiérarchie de rangs, refus d'escalade de privilège) qui portent à elles seules le modèle d'autorisation de la plateforme, et sur lesquelles repose la mesure **A01 — Broken Access Control** du chapitre sécurité. Les 44 cas couvrent notamment le contournement propriétaire, le refus d'accorder une permission que l'on ne détient pas soi-même, et l'interdiction d'agir sur un rôle de rang égal ou supérieur au sien.

Limites assumées (pistes d'amélioration)

- **Contrôleurs non couverts** (0 %) : la convention du projet réserve les contrôleurs aux tests e2e (`*.e2e-spec.ts`), base réelle via testcontainer), qui **ne sont pas encore écrits**. C'est le principal poste de progression du chiffre global, et la validation de bout en bout repose aujourd'hui sur la recette manuelle.
- **Frontend** : `apps/web` n'a pas encore de tests unitaires (`vitest --passWithNoTests`). Trois des dix anomalies du registre (BUG-07, BUG-08, BUG-09) vivent dans `apps/web` — c'est le point aveugle du harnais, et la priorité suivante : composables et stores Pinia.
- `servers` , `members` , `users` , `channels` restent sous 60 % : ces services sont les plus anciens et ont été écrits avant la généralisation du harnais.

Le seuil en CI est volontairement calé **au niveau atteint** plutôt qu'à une valeur cible : il fonctionne comme un cliquet, garantissant que la couverture ne peut que progresser, et sera relevé à chaque lot de tests ajouté.



Sécurité — couverture OWASP Top 10

Compétence C2.2.3

Livrable RNCP **C2.2.3** (volet sécurité) — Présentation des mesures de sécurité mises en œuvre.

Critère : les mesures prises permettent de couvrir les 10 failles de sécurité principales décrites par l'OWASP.

Ce document présente, catégorie par catégorie, les mesures en place dans NookApp et, en toute transparence, les points restants à durcir.

Tableau de synthèse

#	CATÉGORIE OWASP 2021	COUVERTURE	MESURE PRINCIPALE
A01	Broken Access Control	Forte	Chaîne de guards + scoping multi-tenant
A02	Cryptographic Failures	Bonne	Better Auth (hash), sessions DB, TLS Traefik
A03	Injection	Très forte	ORM paramétré (Drizzle) + validation Zod
A04	Insecure Design	Bonne	Multi-tenancy, rate limiting, vérif. email
A05	Security Misconfiguration	Partielle	Helmet + CORS verrouillé (HSTS à ajouter)
A06	Vulnerable Components	Bonne	Dependabot + lockfile figé
A07	Identification & Auth Failures	Bonne	Better Auth + rate limit ciblé
A08	Software & Data Integrity	Bonne	CI, images immuables, commitlint
A09	Logging & Monitoring	Partielle	Sentry (exceptions API + navigateur, release par SHA) ; journal d'audit et métriques à venir
A10	SSRF	Partielle	Validation d'URL + origines de confiance

A01 — Broken Access Control

Contrôle d'accès à **trois niveaux**, appliqué de façon déclarative sur les routes :

- **AuthGuard** (`auth/auth.guard.ts`) — vérifie la session Better Auth, sinon `401`.
- **ServerScopeGuard** (`members/server-scope.guard.ts`) — vérifie que l'utilisateur est **membre** du serveur ciblé (`serverId` + `userId`) et injecte `req.server` / `req.member`. Empêche toute requête inter-tenant par construction.
- **PermissionsGuard** — permissions granulaires (bitfield) via `@RequirePermission(PERMISSIONS.ManageMembers)`. Le propriétaire a toutes les permissions.

Contrôles d'ownership dans les services (suppression de serveur, modération : pas d'auto-kick, protection du propriétaire, hiérarchie des rangs). Système de **ban** (`server_ban`) vérifié aussi à l'entrée par invitation.

À **renforcer** : les handlers Socket.IO valident l'appartenance *manuellement* (`isMember()`) plutôt que par un guard automatique — risque d'oubli lors de l'ajout d'un nouvel événement.

A02 — Cryptographic Failures

- **Mots de passe** : hachés par Better Auth ; minimum **12 caractères** imposé par Zod (`passwordSchema`).
- **Sessions** : stockées en base (`session`), avec expiration ; cascade delete à la suppression du compte.
- **Transport** : HTTPS de bout en bout via Traefik + Let's Encrypt ; les URLs de prod sont en `https://`. `trust proxy` activé (API derrière Traefik).
- **Secrets** : `BETTER_AUTH_SECRET` (≥ 32 c), `JWT_SECRET` — hors dépôt (`.env` gitignoré, secrets GitHub Actions / VPS).

À **renforcer** : en-tête **HSTS** non explicitement forcé ; `JWT_SECRET` partagé entre Better Auth et le token Hocuspocus (rotation moins souple).

A03 — Injection

Point le plus solide de l'application.

- **Aucune requête SQL brute** : tout passe par l'ORM **Drizzle**, qui paramètre les requêtes (`.where(and(eq(member.serverId, serverId), ...))`).
- **Validation Zod systématique aux frontières** (`ZodPipe` global) : email (`.email().max(254)`), mot de passe, slug (`/^[a-z0-9-]+$`), longueur des messages (≤ 4000), couleurs hex, etc.
- **Anti-XSS côté emails** : les valeurs utilisateur injectées dans les mails sont échappées (test unitaire dédié).

A04 — Insecure Design

- **Multi-tenancy par conception** : `serverId` sur chaque table possédée, scoping obligatoire (cf. A01).
- **Rate limiting** : global `@nestjs/throttler` (300 req/min) + surcharges par endpoint (ex. lookup DM : 20 req/min).

- **Vérification d'email obligatoire** (`requireEmailVerification: true`) : bride les inscriptions automatisées.
- **Invitations** : usage limité + expiration.

À renforcer : journal d'audit des actions sensibles (cf. A09) ; anti- énumération d'utilisateurs sur le lookup DM.

A05 — Security Misconfiguration

- **Helmet** (`main.ts`) : en-têtes de sécurité par défaut (`X-Content-Type-Options` , `X-Frame-Options` , etc.). CSP volontairement désactivée pour l'API (contexte CORS).
- **CORS verrouillé** : origine exacte `NEXT_PUBLIC_WEB_URL` (HTTP et Socket.IO), `credentials: true` .
- **Secrets hors image ; tags Docker immuables** (`<branche>-<sha>`) — pas de `latest` en déploiement applicatif.

À renforcer : ajouter `Strict-Transport-Security` (HSTS) via Helmet ; épingler la version de l'image LiveKit (actuellement `latest`).

A06 — Vulnerable and Outdated Components

- **Dependabot** (`.github/dependabot.yml`) : npm hebdomadaire (mises à jour mineures/patch groupées), GitHub Actions mensuel.
- **Lockfile figé** en CI (`pnpm install --frozen-lockfile`) : pas de dérive.
- **Dépendances récentes** : NestJS 10, Better Auth, Drizzle, Helmet 8, Socket.IO 4.8, Zod 4, Node 22 LTS.

À renforcer : pas encore de SBOM ni de `npm audit` bloquant en CI.

A07 — Identification and Authentication Failures

- **Better Auth** : email + mot de passe, **vérification d'email** obligatoire, `autoSignIn: false` .
- **Unicité** : index uniques sur `email` et `username` ; validation du username (trim + lowercase) avant insertion.
- **Rate limit ciblé sur l'auth** : `/sign-in/email` 5/min, `/sign-up/email` 5/min, `/forget-password` 3/min.
- **Réinitialisation de mot de passe** par lien email sécurisé ; **suppression de compte** avec transfert de propriété des Nooks (RGPD).

À renforcer : pas de 2FA ; un seul mode d'auth (pas d'OAuth externe activé en V1, contrairement à ce qui était initialement envisagé) ; IP/User-Agent de session stockés mais non contrôlés.

A08 — Software and Data Integrity Failures

- **CI stricte** avant fusion (lint, typecheck, test, build) — gate `ci-success` .
- **Commits normés** : commitlint (Conventional Commits) + Husky/lint-staged.
- **Artefacts immuables** : chaque image Docker taguée par SHA de commit ; déploiement par tag immuable → rejouable et traçable.

- **Migrations** : versionnées (drizzle-kit), appliquées de façon déterministe au démarrage.

À renforcer : signature d'artefacts (cosign) et provenance SLSA non mises en place.

A09 — Security Logging and Monitoring Failures

Capture centralisée des exceptions — @sentry/node côté API,

JavaScript remonte automatiquement, avec sa trace et le **SHA du commit déployé** (`SENTRY_RELEASE` = tag d'image immuable), dès lors que `SENTRY_DSN` est renseigné sur la stack. Les 4xx sont filtrées pour ne pas noyer les anomalies réelles. Les environnements staging et production sont séparés par

Ce point était le plus faible du chapitre ; il a été traité après le constat que deux anomalies du registre (BUG-08, BUG-09) n'avaient été découvertes que par des retours d'usage.

Reste à renforcer : pas de **journal d'audit** des événements de sécurité (connexions échouées, refus de permission, suppressions) — Sentry capture les *erreurs*, pas les *événements métier sensibles*. La journalisation centralisée et les métriques (Prometheus, Loki, Grafana, alerting) relèvent du **système de supervision du Bloc 4 (C4.1.2)**. À court terme : middleware de log des requêtes HTTP + journal d'audit des actions de modération et de suppression.

A10 — Server-Side Request Forgery (SSRF)

Surface d'attaque limitée (peu d'appels sortants pilotés par l'utilisateur).

- **Validation d'URL** : `iconUrl` validé par Zod (`.url()`).
- **Origines de confiance** Better Auth (`trustedOrigins`) : évite les redirections ouvertes dans les callbacks d'auth.
- **Hocuspocus** écoute sur un port interne, derrière Traefik.

À renforcer : valider `WEB_URL` / `BETTER_AUTH_URL` au démarrage (éviter un

Conclusion

NookApp intègre la sécurité **dès la conception** : contrôle d'accès à plusieurs niveaux, isolation multi-tenant par construction, validation systématique (anti-injection excellente), authentification robuste, chaîne CI/CD intègre. Les axes de durcissement identifiés — **journalisation/supervision (A09)**, HSTS (A05), validation d'environnement (A10) — sont explicitement tracés ; la supervision relève du Bloc 4. Aucune des 10 catégories n'est laissée sans mesure.

Compétence C2.2.3

Livrable RNCP **C2.2.3** (volet accessibilité) — Présentation des actions mises en œuvre pour permettre l'accès à l'application aux personnes en situation de handicap.

Critères : le référentiel d'accessibilité choisi est présenté et justifié ; le prototype répond aux exigences du référentiel préalablement établi.

1. Référentiel choisi et justification : OPQUAST

Le référentiel retenu est **OPQUAST** (Open Quality Standards, 240 règles qualité web), plutôt que le **RGAA**.

Justification :

- **Nature du produit.** NookApp comprend un **monde 2D rendu dans un canvas Phaser**, intrinsèquement non restituable par un lecteur d'écran. Viser une conformité **RGAA/WCAG AA totale** sur *l'ensemble* de l'application serait irréaliste et malhonnête : le jeu lui-même ne peut pas être rendu accessible au clavier/lecteur d'écran sans le dénaturer.
- **Périmètre pertinent.** OPQUAST permet de **cibler les interfaces autour du canvas** — authentification, chat, menus, réglages, marketplace — là où l'accessibilité a un vrai sens et un vrai impact.
- **Approche qualité.** OPQUAST couvre l'accessibilité **et** la qualité d'usage (i18n, formulaires, navigation), cohérent avec un projet évalué sur sa qualité globale.

Le RGAA reste la référence réglementaire (services publics) ; il est cité comme horizon pour les écrans hors-jeu, mais OPQUAST est le référentiel **appliqué et mesuré** ici.

2. Périmètre d'application

ZONE	ACCESSIBILITÉ VISÉE	ÉTAT MESURÉ
Auth (login, register, reset)	Complète (formulaires, clavier, langue)	cf. grille §5
Chat, DM, menus, réglages, modales	Élevée (sémantique, ARIA, focus)	cf. grille §5 — 4 non-conformités sur le focus des modales
Monde 2D (canvas Phaser)	Hors périmètre — alternative : les mêmes actions (chat, salons, membres) restent accessibles via les panneaux DOM.	

3. État des lieux initial (audit)

Un audit du frontend a établi une base **partiellement conforme** : bonne sémantique HTML (`main`, `nav`, `header`, `section`, boutons typés), contrastes gérés par tokens de thème, formulaires d'auth labellisés (`<label for>`), navigation clavier sur les cartes de salon. Lacunes structurelles identifiées : **pas d'attribut `lang`**, **pas de lien d'évitement**, **modales sans `role="dialog" / aria-modal`**, focus clavier peu visible, pas de prise en compte de `prefers-reduced-motion`.

4. Actions mises en œuvre

Correctifs appliqués pour répondre aux règles OPQUAST prioritaires :

#	ACTION	FICHIER(S)	RÈGLE
1	Attribut <code>lang</code> dynamique sur <code><html></code> , suivant la locale FR/EN active	<code>app.vue</code> (<code>useHead</code>)	OPQUAST 20 / RGAA 8.3
2	Lien d'évitement « Aller au contenu principal » (visible au focus clavier), cible <code>#main-content</code> posée sur les layouts	<code>app.vue</code> , <code>layouts/default.vue</code> , <code>layouts/app.vue</code> , <code>main.css</code>	OPQUAST 91 / RGAA 12.13
3	<code>role="dialog"</code> + <code>aria-modal="true"</code> + <code>aria-label</code> sur les modales (invitation, création de Nook, édition salon/catégorie)	<code>InviteModal</code> , <code>EditChannelModal</code> , <code>EditCategoryModal</code> , <code>pages/app/index.vue</code>	OPQUAST 143
4	Focus clavier toujours visible : style <code>:focus-visible</code> global (contour net)	<code>main.css</code>	OPQUAST 96 / RGAA 10.7
5	Respect de <code>prefers-reduced-motion</code> : animations/transitions neutralisées si l'utilisateur le demande	<code>main.css</code>	OPQUAST 141

Ces correctifs s'ajoutent à l'existant : sémantique HTML, labels de formulaires, internationalisation, primitives partiellement issues de Reka UI. L'état de conformité point par point est établi au §5.

Extraits

Attribut de langue suivant la locale :

```
// apps/web/app.vue
const { locale } = useI18n();
useHead({ htmlAttrs: { lang: () => locale.value } });
```

Lien d'évitement (masqué jusqu'au focus) :

```
/* apps/web/assets/css/main.css */
.skip-link { transform: translateY(-150%); }
.skip-link:focus { transform: translateY(0); }
```

5. Grille de contrôle et score de conformité

La conformité n'est pas déclarée par thème mais **contrôlée point par point**. Le tableau ci-dessous liste les **30 points de contrôle OPQUAST applicables** au périmètre défini au §2 (interfaces DOM hors canvas). Chaque ligne porte son état et sa preuve.

La colonne **État** distingue trois situations, et cette distinction est volontairement stricte : un point n'est déclaré conforme que si une preuve dans le code source l'établit. Les points qui dépendent d'un test manuel non encore réalisé sont marqués « à vérifier » plutôt que supposés conformes.

#	POINT DE CONTRÔLE	ÉTAT	PREUVE
1	Langue principale déclarée sur <code><html></code>	Conforme	<code>app.vue</code> — <code>useHead({ htmlAttrs: { lang } })</code>
2	Langue mise à jour au changement de locale	Conforme	<code>locale</code> réactive FR/EN
3	Titre de page pertinent et unique par vue	À rejouer	revue page par page à faire
4	Structure sémantique (<code>header</code> , <code>nav</code> , <code>main</code>)	Conforme	<code>layouts/default.vue</code> , <code>layouts/app.vue</code>
5	Une seule balise <code>main</code> par page	Conforme	<code>#main-content</code> unique par layout
6	Hiérarchie de titres cohérente	À rejouer	revue page par page à faire
7	Lien d'évitement vers le contenu principal	Conforme	<code>app.vue</code> + <code>.skip-link</code> (<code>main.css</code>)
8	Lien d'évitement visible à la prise de focus	Conforme	<code>.skip-link:focus</code> (<code>main.css</code>)
9	Focus clavier toujours visible	Conforme	<code>:focus-visible</code> global (<code>main.css</code>)
10	Ordre de tabulation cohérent avec l'ordre visuel	À rejouer	test clavier à réaliser
11	Tout contrôle actionnable au clavier	À rejouer	test clavier à réaliser
12	Aucun piège au clavier	À rejouer	test clavier à réaliser
13	Modales : <code>role="dialog"</code> + <code>aria-modal</code>	Conforme	<code>InviteModal</code> , <code>EditChannelModal</code> , <code>EditCategoryModal</code>
14	Modales : fermeture par Échap	Non conforme	aucun gestionnaire <code>keydown</code> sur ces trois modales
15	Modales : focus déplacé dans la boîte à l'ouverture	Non conforme	cf. §6
16	Modales : focus contraint dans la boîte	Non conforme	cf. §6
17	Modales : focus restitué à l'élément déclencheur	Non conforme	cf. §6

#	POINT DE CONTRÔLE	ÉTAT	PREUVE
18	Champs de formulaire associés à un <code><label for></code>	Conforme	<code>pages/auth/*.vue</code>
19	Champs obligatoires signalés hors de la seule couleur	À rejouer	revue des formulaires à faire
20	Messages d'erreur de formulaire explicites	À rejouer	revue des formulaires à faire
21	Erreurs liées au champ (<code>aria-invalid</code> / <code>aria-describedby</code>)	Non conforme	absent du code
22	<code>autocomplete</code> sur les champs d'identification	À rejouer	revue des écrans d'auth à faire
23	Contraste texte / fond $\geq 4.5:1$	À rejouer	mesure à réaliser sur les 3 thèmes
24	Contraste des composants d'interface $\geq 3:1$	À rejouer	mesure à réaliser
25	Aucune information portée par la seule couleur	À rejouer	revue des badges à faire
26	Boutons icônes pourvus d'un nom accessible	À rejouer	revue du dock et du HUD à faire
27	Images décoratives neutralisées pour l'assistance	À rejouer	revue à faire
28	Respect de <code>prefers-reduced-motion</code>	Conforme	<code>main.css</code>
29	Zones de mise à jour dynamique annoncées (<code>aria-live</code>)	Non conforme	absent du code
30	Alternative DOM aux fonctions du canvas	Conforme	chat, salons, membres accessibles hors du monde 2D

État au moment de la remise : 11 points conformes et prouvés par le code, 6 points non conformes et identifiés, 13 points restant à vérifier par un test manuel.

Les six non-conformités se concentrent sur **deux sujets** : la gestion du clavier et du focus dans les modales (points 14 à 17) et l'annonce des mises à jour dynamiques aux technologies d'assistance (points 21 et 29). Elles sont détaillées au §6.

Honnêteté de la mesure. Les 13 points « à vérifier » ne sont pas des points défaillants : ce sont des points dont la conformité est probable mais qu'aucune preuve statique ne permet d'établir, et qui exigent un test manuel (navigation clavier réelle, mesure de contraste à l'outil, revue page par page). Les déclarer conformes sans les avoir testés reviendrait à produire un chiffre de conformité non fondé. Le point 14 illustre exactement ce risque : il était supposé conforme lors de l'audit initial, la relecture du code a montré l'inverse.

Conséquence pour la suite. Deux actions sont prioritaires : réaliser la campagne de tests manuels sur les 13 points restants, et brancher `axe-core` sur la CI pour que la partie automatisable de la grille soit rejouée à chaque livraison. Aujourd'hui, aucune régression d'accessibilité ne serait détectée automatiquement.

6. Pistes d'amélioration (transparence)

- **Piège de focus** complet dans les modales (maintenir le focus dans la boîte de dialogue) — les primitives `Dialog` de Reka UI, déjà installées, sont le chemin naturel.
- `aria-invalid` / `aria-describedby` sur les champs de formulaire en erreur.
- `aria-live` sur le flux de messages entrants du chat.
- **Tests automatisés** `axe-core` intégrés à la CI pour non-régression.

Ces points sont identifiés et priorisés. Ils ne remettent pas en cause la démarche, mais la conformité OPQUAST n'est pas encore établie sur l'ensemble du périmètre : 11 points sur 30 sont prouvés à la date de remise, 6 sont non conformes et 13 restent à vérifier.

Compétence C2.2.4

Livrable RNCP **C2.2.4** — Déployer le logiciel à chaque modification de code et de façon progressive [...] afin de présenter une solution stable et conforme à l'attendu.

Livrables : *l'historique des différentes versions + la dernière version du logiciel fonctionnel, fiable et viable*. Critères : *un système de gestion de versions est utilisé ; les évolutions du prototype sont tracées ; le logiciel est fonctionnel et manipulable en autonomie par un utilisateur*.

1. Système de gestion de versions

- **Contrôle de version** : **Git** (dépôt GitHub). 164 commits, une branche par ticket, fusion par Pull Request vers `dev` puis `main` (jamais de commit direct sur les branches protégées).
- **Convention de messages** : **Conventional Commits** (`feat`, `fix`, `chore`, `docs`, `refactor`, `test`, `ci` ...), imposée par **commitlint** via un hook Husky `commit-msg`. Chaque évolution est donc qualifiée par son type et son scope, de façon machine-lisible.
- **Versionnage sémantique** : **SemVer** (`MAJOR.MINOR.PATCH`).

2. Traçabilité automatisée des versions (release-please)

Les évolutions sont tracées automatiquement, sans étape manuelle, grâce à **release-please** :

- Workflow : `.github/workflows/release-please.yml` (déclenché sur push `main`).
- Config : `release-please-config.json` + manifeste `.release-please-manifest.json`.

Fonctionnement :

```
Commits Conventional Commits sur main
|
▼
release-please ouvre/maj une "release PR"
├─ calcule le prochain numéro SemVer (feat → minor, fix → patch, ! → major)
├─ met à jour CHANGELOG.md (regroupé par Fonctionnalités / Corrections...)
└─ bump la version dans package.json
    │ (au merge de la release PR)
    ▼
    crée le tag vX.Y.Z + la GitHub Release
```

Le journal des versions (`CHANGELOG.md`) est ainsi dérivé directement de l'historique Git : **aucune évolution ne peut échapper au changelog** si elle est commitée dans les règles.

État réel à la remise. La configuration est en place et commitée (`chore(release): add release-please versioning setup`), mais le workflow ne se déclenche que sur `main` : la **première release automatisée n'a pas encore été produite**. L'entrée `1.0.0` du `CHANGELOG.md` a donc été rédigée manuellement, et le tag `v1.0.0` sera créé par `release-please` au premier merge de la release PR. Ce décalage est assumé : il illustre que le mécanisme de versionnage a été outillé avant d'être exercé, et non l'inverse.

3. Historique des versions

La version courante est la `1.0.0` (première version viable / V1). Le détail figure dans `CHANGELOG.md`.
Grandes étapes du prototype (dérivées de l'historique Git réel) :

PÉRIODE	JALON	CONTENU
2026-05	Fondations & monde	Monorepo, NestJS, Nuxt, Drizzle, Better Auth ; monde Phaser (LimeZu), multijoueur, éditeur de map.
2026-05	Industrialisation	Images Docker de prod, déploiement continu vers le VPS, routage Traefik.
2026-05 → 06	Expérience produit	Refonte UI (thèmes, sidebar+dock, HUD), salons thématiques, notifications, messages privés .
2026-06	Durcissement & conformité	Sécurité (rate limit, Helmet, CORS), sauvegardes PostgreSQL , contrôles RGPD , améliorations d'auth (username unique, emails).
2026-06	1.0.0	Consolidation V1 : plateforme fonctionnelle et déployée.

4. Déploiement progressif (lien avec la traçabilité)

Chaque version fusionnée est déployée **progressivement** : d'abord sur **staging** (`dev` → <https://dev.nookapp.eu>), puis en **production** (`main` → <https://nookapp.eu>). Chaque image Docker est taguée par le **SHA du commit** (`<branche>-<sha>`), ce qui relie de façon immuable *une version déployée à un point précis de l'historique*, et rend tout retour arrière trivial. Détail : « Protocole de déploiement continu ».

5. Dernière version — fonctionnelle, fiable, viable

- **Fonctionnelle** : la V1 est en ligne et manipulable **en autonomie** par un utilisateur (inscription → création de Nook → invitation → chat → monde). Le parcours est décrit dans le « Manuel d'utilisation ».
- **Fiable** : chaque version passe la CI complète (lint/typecheck/test/build) et un **healthcheck bloquant** après déploiement.
- **Viable** : déploiement reproductible (images immuables + migrations automatiques), sauvegardes de base automatisées, retour arrière par redéploiement d'un tag antérieur.



Cahier de recettes

Compétence C2.3.1

Livrable RNCP **C2.3.1** — Élaborer le cahier de recettes en rédigeant les scénarios de tests et les résultats attendus afin de détecter les anomalies de fonctionnement et les régressions éventuelles.

Critères : le cahier reprend l'ensemble des fonctionnalités attendues ; les tests fonctionnels, structurels et de sécurité exécutés sont conformes au plan défini.

1. Objectif et méthode

Ce cahier définit les scénarios de recette permettant de valider, avant chaque mise en production, que NookApp fonctionne conformément à l'attendu. Il couvre trois familles de tests :

- **Fonctionnels** (F) — parcours utilisateur de bout en bout.
- **Structurels** (S) — automatisés (CI) : lint, typage, tests unitaires, build.
- **Sécurité** (SEC) — contrôles d'accès, validation, isolation multi-tenant.

Convention : chaque cas a un identifiant, des préconditions, des étapes, un résultat attendu et un statut :

STATUT	SIGNIFICATION
Conforme	Conforme au premier passage.
✗ → ✓	Non conforme au premier passage , anomalie ouverte, corrigée, cas rejoué et conforme. La référence du bogue est indiquée.
À rejouer	Non rejoué sur la campagne courante.

Métadonnées d'exécution de la campagne

CHAMP	VALEUR
Environnement	staging — https://dev.nookapp.eu (jeu de données de test)
Version sous test	à renseigner à l'exécution (<code>IMAGE_TAG</code> = <code>dev-<sha12></code>)
Date de campagne	à renseigner à l'exécution
Testeur	à renseigner à l'exécution
Navigateur	Chrome / Firefox desktop, version courante
Comptes utilisés	3 comptes de test (propriétaire, admin, membre simple)

Un cas en échec ne se corrige jamais « sur place » : il ouvre une fiche d'anomalie, qui suit le pipeline décrit au chapitre « Plan de correction des bogues », et le cas n'est repassé en  qu'après re-jeu sur staging. Les cinq cas marqués  →  ci-dessous documentent ce cycle sur des anomalies réellement survenues.

2. Tests structurels (automatisés — CI)

ID	CONTRÔLE	COMMANDE / MÉCANISME	RÉSULTAT ATTENDU
S-01	Style de code	<code>pnpm lint</code> (job CI <code>lint</code>)	0 erreur ESLint
S-02	Typage strict	<code>pnpm typecheck</code> (job <code>typecheck</code>)	0 erreur <code>tsc</code>
S-03	Tests unitaires	<code>pnpm test</code> (job <code>test</code>)	152/152 au vert
S-04	Build complet	<code>pnpm build</code> (job <code>build</code>)	API + Web compilent
S-05	Gate d'intégration	job <code>ci-success</code>	vert = fusion autorisée
S-06	Format des commits	commitlint (hook)	message conforme sinon rejet
S-07	Healthcheck déploiement	<code>GET /api/v1/health</code>	HTTP 200, <code>db: true</code>

3. Tests fonctionnels

3.1 Authentification & compte

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-A01	Inscription	Renseigner email + username + mot de passe (≥ 12 c.) → valider	Compte créé, email de vérification envoyé, pas de connexion auto	Conforme
F-A02	Vérification email	Cliquer le lien reçu	Email vérifié, connexion possible	Conforme
F-A03	Mot de passe trop court	Saisir un mot de passe < 12 c.	Message d'erreur, inscription refusée	Conforme
F-A04	Email déjà utilisé	S'inscrire avec un email existant	Message générique + email de notification au compte existant	 →  BUG-01
F-A05	Username déjà pris	S'inscrire avec un username existant	Erreur « nom d'utilisateur indisponible »	Conforme
F-A06	Connexion	Email + mot de passe valides	Redirection vers l'app	Conforme
F-A07	Réinitialisation mot de passe	Demander un reset → lien email → nouveau mot de passe	Mot de passe changé, connexion OK	Conforme
F-A08	Suppression de compte (RGPD)	Réglages → supprimer (transfert des Nooks possédés)	Compte + sessions supprimés, propriété transférée	Conforme

3.2 Nooks (serveurs) & membres

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-N01	Créer un Nook	Bouton « Nouveau Nook » → nom → créer	Nook créé, map par défaut, redirection	Conforme
F-N02	Générer une invitation	Réglages → inviter → copier le lien	Lien d'invitation valide généré	Conforme
F-N03	Rejoindre via invitation	Ouvrir le lien avec un autre compte	Ajout comme membre, accès au Nook	Conforme
F-N04	Invitation invalide	Ouvrir un code inexistant	Erreur « invitation introuvable »	Conforme
F-N05	Kick d'un membre	Membres → kick (avec permission)	Membre retiré du Nook	Conforme
F-N06	Bannir un membre	Membres → bannir	Membre retiré + ajouté aux bannis	Conforme
F-N07	Ré-invitation d'un banni	Banni tente de rejoindre via lien	Accès refusé (banni)	Conforme
F-N08	Débannir	Bannis → débannir	Peut de nouveau rejoindre	Conforme

3.3 Salons & messagerie

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-M01	Créer un salon	Avec permission ManageChannels	Salon texte créé et listé	Conforme
F-M02	Envoyer un message	Saisir + Entrée	Message affiché en temps réel chez tous les membres	Conforme
F-M03	Éditer son message	Éditer un message dont on est l'auteur	Contenu mis à jour, marque « édité »	Conforme
F-M04	Éditer le message d'autrui	Tenter d'éditer le message d'un autre	Action refusée	Conforme
F-M05	Supprimer (modération)	Non-auteur avec ManageMessages	Message supprimé	Conforme
F-M06	Message privé (DM)	Rechercher un utilisateur par handle → écrire	Conversation créée, message livré	Conforme

3.4 Monde 2D & temps réel

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-W01	Déplacement	Entrer dans le monde, ZQSD ou flèches	Personnage se déplace, animation correcte	Conforme
F-W02	Multijoueur	Deux joueurs dans le même Nook	Chacun voit l'autre bouger en temps réel	Conforme
F-W03	Voix de proximité	Se rapprocher d'un joueur dans une zone vocale	Connexion audio LiveKit établie	Conforme
F-W04	Éditeur de map	Admin : poser sols/pièces/décor	Modifications visibles et persistées	Conforme
F-W05	Édition collaborative	Deux admins éditent la map simultanément	Fusion sans conflit (CRDT)	Conforme
F-W06	Déconnexion en mouvement	Un joueur quitte le Nook pendant qu'il se déplace	Sprite retiré, aucun personnage résiduel	 →  BUG-09
F-W07	Changement rapide de salon vocal	Enchaîner trois salons vocaux sans attendre	Une seule salle active en fin de séquence	 →  BUG-08
F-W08	Reconnexion de l'éditeur	Couper le réseau > expiration du jeton, puis rétablir	Reconnexion Hocuspocus et resynchronisation effectives	 →  BUG-10

3.5 Interface & accessibilité

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-U01	Changer de thème	Basculer clair/sombre/pixel	Thème appliqué et persistant	Conforme
F-U02	Changer de langue	FR ↔ EN	Interface traduite, <code><html lang></code> mis à jour	Conforme
F-U03	Lien d'évitement	Tabulation dès le chargement	Lien « Aller au contenu » visible, saut au contenu	Conforme
F-U04	Focus clavier	Naviguer au clavier	Élément focalisé toujours visible	Conforme
F-U05	Sélection d'un salon au clavier	Focaliser un salon → Entrée / Espace	Salon ouvert, identique au clic souris	 →  BUG-07
F-U06	Personnalisation d'avatar	Réglages → éditeur de personnage → enregistrer	Apparence persistée et visible des autres joueurs	À rejouer
F-U07	État de lecture	Recevoir un message dans un salon non ouvert	Salon marqué non-lu, remis à zéro à l'ouverture	À rejouer

3.6 Rôles & permissions

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-R01	Créer un rôle	Réglages → rôles → nouveau rôle + permissions	Rôle créé et listé	À rejouer
F-R02	Assigner un rôle	Membres → attribuer le rôle à un membre	Permissions du rôle effectives immédiatement	À rejouer
F-R03	Retirer une permission	Retirer <code>ManageChannels</code> du rôle	Le membre ne peut plus créer de salon (403)	À rejouer
F-R04	Hiérarchie des rôles	Membre tente d'agir sur un rang égal ou supérieur	Action refusée	À rejouer
F-R05	Supprimer un rôle	Supprimer un rôle assigné	Rôle retiré des membres, permissions recalculées	À rejouer

3.7 Catégories de salons

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-C01	Créer une catégorie	Avec permission ManageChannels	Catégorie créée et listée	À rejouer
F-C02	Ranger un salon	Déplacer un salon dans une catégorie	Salon rattaché, ordre persisté	À rejouer
F-C03	Renommer / personnaliser	Modifier nom, couleur, icône	Modifications visibles de tous les membres	À rejouer
F-C04	Supprimer une catégorie	Supprimer une catégorie non vide	Salons conservés et détachés	À rejouer

3.8 Données personnelles (RGPD)

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
F-G01	Export des données	Réglages → exporter mes données	Archive téléchargeable contenant compte, messages, appartenances	À rejouer
F-G02	Suppression de compte	Cf. F-A08	Données supprimées, propriété des Nooks transférée	Conforme

4. Tests de sécurité

ID	SCÉNARIO	ÉTAPES	RÉSULTAT ATTENDU	STATUT
SEC-01	Accès non authentifié	Appeler une route protégée sans session	HTTP 401	Conforme
SEC-02	Isolation multi-tenant	Membre du Nook A appelle une ressource du Nook B	HTTP 403 (non membre)	Conforme
SEC-03	Escalade de privilège	Membre sans permission tente de kick/ban	HTTP 403	Conforme
SEC-04	Protection du propriétaire	Tenter de kick le propriétaire	Refusé	Conforme
SEC-05	Validation d'entrée	Envoyer un payload invalide (email/slug/longueur)	Rejet Zod, HTTP 4xx	Conforme
SEC-06	Rate limit auth	> 5 tentatives de connexion/min	Requêtes bridées	Conforme
SEC-07	Injection SQL	Payloads malicieux dans les champs	Aucun effet (ORM paramétré)	Conforme
SEC-08	XSS via email	Nom contenant <code><script></code>	Sortie HTML échappée	Conforme
SEC-09	En-têtes de sécurité	Inspecter les réponses HTTP	En-têtes Helmet présents	Conforme
SEC-10	CORS	Requête depuis une origine non autorisée	Bloquée	Conforme

5. Suivi des régressions

- **Automatique** : la CI rejoue S-01 → S-05 à chaque PR ; toute régression sur le cœur métier (auth, permissions, modération, messages) casse un test unitaire et bloque la fusion.
- **Manuel** : les cas F et SEC sont rejoués sur staging avant chaque promotion vers la production.
- Les anomalies détectées alimentent le « Plan de correction des bogues ».

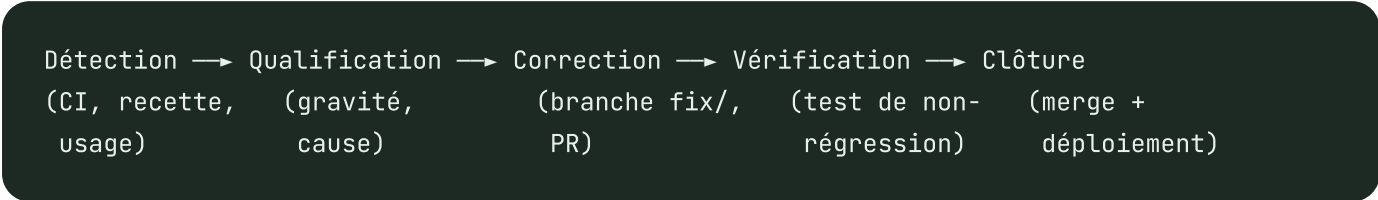
9 Plan de correction des bogues

Compétence C2.3.2

Livrable RNCP **C2.3.2** — Élaborer un plan de correction des bogues à partir de l'analyse des anomalies et des régressions détectées au cours de la recette afin de garantir le fonctionnement du logiciel conformément à l'attendu.

Critères : les bogues sont détectés, qualifiés et traités ; une analyse des points d'amélioration est réalisée pour chaque test en échec ; les corrections proposées sont conformes à l'attendu.

1. Processus de traitement d'une anomalie



- **Détection** : échec CI, cas de recette en échec, retour d'usage.
- **Qualification** : gravité (bloquant / majeur / mineur), périmètre, cause racine (pas seulement le symptôme).
- **Correction** : branche `fix/ ...`, correctif à la **racine** (là où tous les appelants passent), Conventional Commit `fix(scope): ...`.
- **Vérification** : ajout/mise à jour d'un **test de non-régression** quand c'est pertinent, CI verte, re-test du cas de recette.
- **Clôture** : squash-merge, déploiement progressif (staging → prod).

Échelle de gravité

NIVEAU	DÉFINITION	SLA CIBLE
Bloquant	Empêche un parcours principal ou fuite de sécurité	Immédiat
Majeur	Fonctionnalité dégradée, contournement possible	Avant prochaine release
Mineur	Cosmétique / confort	Planifié

2. Registre des anomalies traitées

Anomalies réelles détectées et corrigées au cours du développement et du durcissement V1.

BUG-01 — Ordre de validation à l'inscription (majeur, sécurité/UX)

- **Détection** : recette du parcours d'inscription (cas type F-A04/F-A05).

- **Symptôme** : un email déjà enregistré pouvait déclencher d'abord l'erreur d'unicité du **username**, masquant le fait que l'**email** existait déjà.
- **Cause racine** : le contrôle d'unicité username s'exécutait avant celui de l'email dans le hook d'inscription.
- **Correction** : inversion de l'ordre — vérification « email déjà utilisé » **avant** l'unicité du username.
- **Vérification** : re-test F-A04/F-A05, comportement conforme (notification à l'email existant).
- **Point d'amélioration** : dans un hook d'inscription, l'ordre des contrôles d'unicité porte du sens fonctionnel et ne doit pas dépendre de l'ordre d'écriture du code. La contrainte est désormais explicitée par un commentaire et verrouillée par le cas de recette F-A04.
- **Statut** : ☒ Corrigé.

BUG-02 — Régression de test sur le join d'un utilisateur banni (majeur)

- **Détection** : échec CI du job `test` (`servers.service.spec.ts`) après l'ajout de la vérification de ban à l'entrée par invitation.
- **Symptôme** : le test attendait une `ConflictException` mais la nouvelle requête de contrôle de ban modifiait la séquence d'appels à la base.
- **Analyse du test en échec** : le **mock** de la base ne simulait que 2 appels (invitation, membre) alors que le code en fait désormais 3 (invitation, **ban**, membre). Le test était périmé, pas le code.
- **Correction** : mise à jour du mock pour refléter la séquence à 3 requêtes.
- **Vérification** : suite complète au vert.
- **Point d'amélioration** : documenter le patron de mock séquentiel pour éviter ce type de fragilité lors d'ajouts de requêtes.
- **Statut** : ☒ Corrigé.

BUG-03 — Avertissement rate limiting « client IP indéterminée » (mineur)

- **Détection** : logs au démarrage en dev (« Rate limiting skipped: could not determine client IP »).
- **Cause racine** : Better Auth cherchait un en-tête `x-forwarded-for` absent en développement local (pas de reverse proxy).
- **Correction** : configuration **dépendante de l'environnement** — `ipAddressHeaders: [ 'x-forwarded-for' ]` uniquement en production (derrière Traefik), rien en local.
- **Vérification** : plus d'avertissement en dev ; rate limit effectif en prod.
- **Point d'amélioration** : une configuration qui suppose un reverse proxy doit être conditionnée à l'environnement plutôt que posée globalement, sans quoi elle échoue silencieusement là où le proxy est absent.
- **Statut** : ☒ Corrigé.

BUG-04 — `MAIL_DRIVER` non configurable (mineur, config)

- **Détection** : préparation du déploiement staging — impossibilité de basculer Mailpit/Resend sans modifier le code.
- **Cause racine** : le driver était instancié **en dur** dans le module mailer. Le choix d'un service externe était donc figé à la compilation, alors qu'il diffère par environnement (Mailpit en dev, Resend en staging/prod).

- **Correction** : sélection par variable d'environnement `MAIL_DRIVER` (Mailpit par défaut), avec erreur explicite si `resend` est demandé sans clé API — échec au démarrage plutôt qu'emails silencieusement perdus.
- **Vérification** : `mailer.module.spec.ts` couvre les trois chemins (Mailpit, Resend valide, Resend sans clé) ; bascule effective validée sur staging.
- **Point d'amélioration** : tout service externe dont le choix diffère selon l'environnement doit être sélectionné par variable d'environnement et échouer au démarrage si sa configuration est incomplète — un email silencieusement perdu coûte plus cher qu'un conteneur qui refuse de démarrer.
- **Clôture** : `2185a5c` — *fix: make MAIL_DRIVER configurable via env (default mailpit)*, 2026-05-26.
- **Statut** :  Corrigé.

BUG-05 — Volume PostgreSQL incompatible (majeur, ops)

- **Détection** : recette de déploiement — le conteneur base refusait de démarrer sur un volume existant.
- **Cause racine** : PostgreSQL 18 a modifié le layout du répertoire de données ; le point de montage du compose, écrit pour la 17, ne correspondait plus. Le tag flottant `postgres:latest` avait fait basculer la version **sans changement de code**.
- **Correction** : épinglage à `postgres:17-alpine`.
- **Vérification** : re-déploiement complet sur staging, volume monté, healthcheck `pg_isready` au vert.
- **Point d'amélioration** : aucun tag d'image flottant dans les fichiers de déploiement — règle appliquée depuis à l'ensemble du compose.
- **Clôture** : `592335f` — *fix: pin postgres to 17-alpine for stable volume layout*, 2026-05-26.
- **Statut** :  Corrigé.

BUG-06 — Clé primaire manquante sur `server_plugin` (majeur, intégrité)

- **Détection** : revue de schéma pendant le durcissement V1.
- **Cause racine** : la table d'association était déclarée sans contrainte d'unicité. Rien n'empêchait deux lignes `(serverId, pluginId)` identiques, donc un plugin activé deux fois pour le même Nook avec deux configurations divergentes — et un comportement dépendant de l'ordre de lecture.
- **Correction** : migration Drizzle en deux temps car des doublons existaient déjà en base — **dédoublonnage** (conservation du `ctid` minimum par couple) **puis** ajout de la clé primaire composite :

```
`` sql DELETE FROM "server_plugin" WHERE ctid NOT IN ( SELECT MIN(ctid) FROM "server_plugin"
GROUP BY "server_id", "plugin_id" ); ALTER TABLE "server_plugin" ADD CONSTRAINT
"server_plugin_pkey" PRIMARY KEY ("server_id", "plugin_id"); ``
```

- **Vérification** : migration rejouée sur staging, insertion en double refusée par la base.
- **Point d'amélioration** : une contrainte d'intégrité manquante ne se voit qu'en revue — les tables d'association sont désormais relues systématiquement.
- **Note sur la traçabilité** : cette anomalie concernait le lot « plugins », retiré du périmètre V1 depuis. La migration n'est donc plus présente dans l'arborescence livrée (les migrations vont de `0000` à `0004`) ; elle reste consultable dans l'historique Git au commit indiqué ci-dessous.
- **Clôture** : `a3ea188` — *fix(db): enforce composite primary key on server_plugin*, 2026-05-21.

- **Statut** : ☒ Corrigé.

BUG-07 — Type d'événement de sélection de salon trop étroit (mineur, a11y)

- **Détection** : `typecheck` en échec après l'ajout de la navigation clavier dans la liste des salons.
- **Cause racine** : `ChannelEntry.vue` déclarait `select: [channel, event: MouseEvent]`. Le composant était donc **typé pour la souris uniquement** : brancher un `keydown` était impossible sans élargir le contrat. Un typage trop étroit bloquait une exigence d'accessibilité.
- **Correction** : élargissement à `MouseEvent | KeyboardEvent` sur l'émetteur et le handler, propagé à `ChannelsList.vue`.
- **Vérification** : `typecheck` au vert, sélection d'un salon au clavier (Entrée / Espace) rejouée — cas de recette F-U05.
- **Clôture** : `d359a84` — *fix(web): widen channel-select event type to include keyboard*, 2026-05-26.
- **Statut** : ☒ Corrigé.

BUG-08 — Course sur le changement rapide de salon vocal (majeur, temps réel)

- **Détection** : usage — en enchaînant rapidement deux salons vocaux, l'utilisateur se retrouvait connecté au **mauvais** salon, ou entendu dans les deux.
- **Cause racine** : `join()` est asynchrone et enchaîne trois `await` (quitter le salon courant, obtenir un jeton LiveKit, se connecter). Rien n'annulait un `join` déjà en vol : un second appel démarrait pendant que le premier était suspendu sur un `await`, et **les deux allaient au bout**. Le dernier à résoudre écrasait `room.value` — l'ordre d'arrivée décidait du résultat, pas l'ordre des clics.
- **Correction** : compteur de séquence `joinSeq` incrémenté à chaque `join / leave`. Chaque `join` capture sa valeur et **abandonne après chaque** `await` si elle a été dépassée ; si la connexion LiveKit a déjà été établie entre-temps, la salle périmée est explicitement déconnectée pour ne pas fuir.
- **Vérification** : enchaînement rapide de trois salons rejoué manuellement, une seule salle active en fin de séquence.
- **Point d'amélioration** : tout enchaînement de plusieurs `await` déclenché par un clic est un candidat à l'annulation — patron réutilisé depuis.
- **Clôture** : `e13f4d8` — *fix(voice): cancel in-flight join via sequence counter*, 2026-05-03.
- **Statut** : ☒ Corrigé.

BUG-09 — Résurrection fantôme d'un joueur déconnecté (majeur, temps réel)

- **Détection** : usage — le sprite d'un joueur ayant quitté le Nook réapparaissait brièvement puis restait figé dans le monde.
- **Cause racine** : course entre deux événements Socket.IO. Un `player:moved` émis **juste avant** la déconnexion pouvait arriver **après** le `player:left` ; `updateRemotePlayer` recréait alors le sprite d'un joueur déjà retiré de la scène, sans que plus aucun événement ne vienne le nettoyer.
- **Correction** : garde à la réception — `player:moved` n'est appliqué que si la scène connaît déjà ce joueur (`scene.hasRemotePlayer`). Un mouvement ne peut plus **créer** un joueur, seulement en déplacer un existant.
- **Vérification** : déconnexions répétées en cours de déplacement, plus aucun sprite résiduel.

- **Point d'amélioration** : sur un transport non ordonné entre types d'événements, les événements de mise à jour ne doivent jamais avoir d'effet de création implicite.
- **Clôture** : `bacae9f` — *fix(world): guard player:moved to prevent ghost respawn*, 2026-05-03.
- **Statut** : ☒ Corrigé.

BUG-10 — Jeton d'édition collaborative périmé à la reconnexion (majeur, sécurité)

- **Détection** : recette de l'éditeur de carte — après une coupure réseau prolongée, la reconnexion Hocuspocus échouait silencieusement et les modifications n'étaient plus synchronisées.
- **Cause racine** : le jeton était récupéré **une seule fois**, avant la construction du provider, puis passé en valeur. Hocuspocus reconnecte tout seul, mais rejouait indéfiniment ce même jeton — expiré. Effet de bord sécurité : un jeton capturé au montage ne reflète plus l'appartenance réelle au serveur, un membre exclu en cours de session gardant un jeton valide jusqu'à expiration.
- **Correction** : passage d'une **fabrique asynchrone** (`token: async () => ...`) au lieu d'une valeur. Hocuspocus la rappelle à chaque tentative de connexion, ce qui redemande un jeton frais à l'API et recontrôle l'appartenance côté serveur à chaque reconnexion.
- **Vérification** : coupure réseau simulée au-delà de l'expiration du jeton, reconnexion et resynchronisation effectives.
- **Point d'amélioration** : un jeton confié à une couche transport qui reconnecte d'elle-même doit toujours être fourni sous forme de fabrique, jamais de valeur — sans quoi la reconnexion rejoue indéfiniment un jeton périmé et contourne le contrôle d'appartenance. Règle appliquée depuis à Hocuspocus et vérifiée sur LiveKit.
- **Clôture** : `c33103b` — *fix(web): refresh hocuspocus token on reconnect*, 2026-05-07.
- **Statut** : ☒ Corrigé.

3. Synthèse & points d'amélioration transverses

ID	GRAVITÉ	DOMAINE	DÉTECTION	COMMIT	STATUT
BUG-01	Majeur	Auth	Recette	feat/auth-already-registered-email	Conforme
BUG-02	Majeur	Tests/CI	Échec CI	feat/v1-hardening	Conforme
BUG-03	Mineur	Sécurité/config	Logs dev	feat/v1-hardening	Conforme
BUG-04	Mineur	Config	Déploiement	2185a5c	Conforme
BUG-05	Majeur	Ops	Recette déploiement	592335f	Conforme
BUG-06	Majeur	Base de données	Revue de schéma	a3ea188	Conforme
BUG-07	Mineur	Accessibilité	Échec typecheck	d359a84	Conforme
BUG-08	Majeur	Temps réel (voix)	Usage	e13f4d8	Conforme
BUG-09	Majeur	Temps réel (monde)	Usage	bacae9f	Conforme
BUG-10	Majeur	Collaboration	Recette	c33103b	Conforme

Répartition des canaux de détection : **3 par la recette**, **2 par la chaîne automatisée** (CI, typecheck), **2 par l'usage**, **1 par revue de schéma**, **1 par les journaux**, **1 en préparation de déploiement**. Chaque canal n'attrape qu'une classe d'anomalies : la chaîne automatisée voit les régressions et les erreurs de typage, la recette voit les écarts fonctionnels, l'usage voit les courses asynchrones qu'aucun test unitaire ne reproduit.

Améliorations préventives retenues :

- **Courses asynchrones** (BUG-08, BUG-09, BUG-10) : la classe d'anomalie la plus représentée, et invisible en test unitaire. Trois règles en découlent — annuler tout enchaînement de `await` déclenché par une action utilisateur, ne jamais laisser un événement de mise à jour créer une entité, ne jamais capturer un jeton par valeur quand la couche transport reconnecte seule.
- **Aucun tag d'image flottant** dans les fichiers de déploiement (BUG-05) : une version de dépendance ne doit jamais changer sans commit.
- **Patron de mock séquentiel documenté** (BUG-02) pour éviter les tests périmés lors de l'ajout d'une requête.
- **Étendre les tests au frontend** (composables/stores) : BUG-07, BUG-08, BUG-09 et BUG-10 vivent tous dans `apps/web`, qui est le point aveugle du harnais — quatre anomalies sur dix.

- **Journalisation/supervision** pour détecter en production les anomalies non couvertes par la recette. C'est ce constat qui a conduit à l'intégration de Sentry (cf. A09 dans « Sécurité — couverture OWASP Top 10 ») ; les métriques système restent cadrées pour le Bloc 4.

4. Garantie de non-régression

Chaque correctif qui touche la logique métier est **verrouillé par un test** rejoué en CI à chaque PR. Le cycle « anomalie → correctif → test → déploiement » s'appuie directement sur le pipeline CI/CD : une correction n'est promue en production qu'après CI verte et healthcheck réussi.

Compétence C2.4.1

Livrable RNCP **C2.4.1** — Documentation technique d'exploitation (manuel de déploiement). Décrit comment installer et mettre en ligne l'application, et justifie les choix de technologies.

1. Architecture cible

NookApp est déployé en **conteneurs Docker** sur un **VPS OVH**, derrière un reverse proxy **Traefik v3** mutualisé (HTTPS automatique via Let's Encrypt). Deux environnements isolés coexistent sur la même machine :

ENVIRONNEMENT	BRANCHE	DOMAINE	STACK	CHEMIN
Staging	dev	dev.nookapp.eu	nookapp-staging	/opt/nookapp-staging/
Production	main	nookapp.eu	nookapp-prod	/opt/nookapp-prod/

Choix technologiques :

- **Docker + Compose** : environnement identique dev/prod, déploiement reproductible.
- **VPS unique** : cohérent avec l'échelle cible (v1) et le budget (~10 €/mois) ; l'architecture permet une montée en charge verticale avant tout rewrite.
- **Traefik** : routage par labels, HTTPS auto, mutualisé avec d'autres projets de l'hôte (pas de proxy dédié à maintenir).
- **GHCR** (GitHub Container Registry) : stockage des images versionnées.

2. Services de la stack (`docker-compose.app.yml`)

SERVICE	IMAGE	RÔLE	PORTS (INTERNE)
web	ghcr.io/<owner>/nookapp-web	Frontend Nuxt (Nitro)	4001
api	ghcr.io/<owner>/nookapp-api	Backend NestJS (HTTP + Socket.IO + Hocuspocus)	4000, 4234
postgres	postgres:17-alpine	Base de données	5432
livekit	livekit/livekit-server	Voix/vidéo WebRTC	7880 (+ TCP/UDP exposés)
db-backup	postgres:17-alpine	Sauvegardes automatisées	—

Volumes persistants : `postgres-data` , `api-uploads` , `db-backups` . Réseaux :

3. Prérequis

- Un VPS Linux avec **Docker** et **Docker Compose v2**.
- **Traefik** en service sur l'hôte, attaché au réseau `dokploy-network` .
- Un enregistrement **DNS** `A` pointant le domaine vers l'IP du VPS.
- Accès **SSH** au VPS.
- Un compte GitHub avec accès au dépôt et à GHCR.

4. Déploiement automatisé (nominal)

Le déploiement est **continu** : il se déclenche à chaque fusion sur `dev` (staging) ou `main` (production). Aucune action manuelle en fonctionnement normal.

1. Fusion d'une PR → push sur `dev` / `main` .
2. GitHub Actions (`cd.yml`) construit les images `api` et `web` , les publie sur GHCR (tags `<branche>` et `<branche>-<sha>`).
3. Actions se connecte en SSH au VPS et exécute : ```bash cd /opt/<stack> export IMAGE_TAG=<branche>-<sha> docker compose -p <stack> -f docker-compose.app.yml --env-file .env pull docker compose -p <stack> -f docker-compose.app.yml --env-file .env up -d --remove-orphans```
4. Le conteneur `api` applique les **migrations Drizzle** au démarrage (`docker-entrypoint.sh`).
5. Actions vérifie `GET https://<domaine>/api/v1/health` (6 tentatives, 10 s).

5. Première installation manuelle d'une stack

Pour provisionner un nouvel environnement (ex. la production) :

```
## 1. Sur le VPS, créer le dossier de la stack
mkdir -p /opt/nookapp-prod && cd /opt/nookapp-prod

## 2. Y déposer docker-compose.app.yml et le fichier .env (voir §6)

## 3. S'authentifier auprès de GHCR
echo "$GHCR_TOKEN" | docker login ghcr.io -u <user> --password-stdin

## 4. Démarrer la stack
export IMAGE_TAG=main-<sha>
docker compose -p nookapp-prod -f docker-compose.app.yml --env-file .env up -d
```

Les migrations s'appliquent automatiquement ; vérifier ensuite le healthcheck.

6. Configuration (fichier `.env`)

Le `.env` de chaque stack vit **uniquement sur le VPS** (`/opt/<stack>/ .env`), jamais dans le dépôt. Gabarit : `.env.production.example`. Variables clés :

VARIABLE	RÔLE
<code>STACK_NAME</code> , <code>DOMAIN</code>	Nom de la stack + domaine (routage Traefik).
<code>IMAGE_TAG</code> , <code>GHCR_OWNER</code>	Version d'image à déployer.
<code>POSTGRES_USER/PASSWORD/DB</code>	Identifiants base.
<code>JWT_SECRET</code> , <code>BETTER_AUTH_SECRET</code>	Secrets d'authentification (≥ 32 c.).
<code>LIVEKIT_API_KEY/SECRET</code> , <code>LIVEKIT_NODE_IP</code>	Voix.
<code>MAIL_DRIVER</code> , <code>RESEND_API_KEY</code> , <code>MAIL_FROM</code>	Emails (Resend en prod).
<code>BACKUP_INTERVAL_HOURS</code> , <code>BACKUP_RETENTION_DAYS</code>	Politique de sauvegarde.

Secrets GitHub (pour la CD) : `VPS_HOST` , `VPS_USER` , `VPS_SSH_KEY` ,

7. Vérification post-déploiement

- `GET https://<domaine>/api/v1/health` → `{ "status": "ok", "db": true }`.
- `docker compose -p <stack> ps` → conteneurs `Up` (postgres `healthy`).
- Documentation d'API disponible sur `https://<domaine>/api/docs`.

8. Sauvegarde & restauration

Le service `db-backup` effectue un `pg_dump` planifié (intervalle et rétention configurables) dans le volume `db-backups`. Restauration via le script

« Manuel de mise à jour ».

9. Retour arrière (rollback)

Redéployer un tag d'image immuable antérieur :

```
cd /opt/<stack>
export IMAGE_TAG=main-<ancien-sha>
docker compose -p <stack> -f docker-compose.app.yml --env-file .env up -d
```

Comme chaque image est liée à un commit précis, le retour à une version connue est immédiat et déterministe.

Compétence C2.4.1

Livrable RNCP **C2.4.1** — Documentation technique d'exploitation (manuel d'utilisation). Décrit comment un utilisateur manipule l'application en autonomie.

1. Qu'est-ce que NookApp ?

NookApp est un espace de travail/convivialité virtuel en 2D. Chaque utilisateur peut créer son **Nook** (son propre espace), y inviter des membres, discuter par messages, et se déplacer dans un monde pixel-art où la **voix se déclenche par proximité** — comme si l'on se rapprochait de quelqu'un dans une pièce.

Accès : <https://nookapp.eu> (production) — navigateur web sur ordinateur.

2. Créer son compte

1. Sur la page d'accueil, cliquer sur **S'inscrire**.
2. Renseigner **email**, **nom d'utilisateur** (unique) et **mot de passe** (12 caractères minimum).
3. Un **email de vérification** est envoyé : cliquer sur le lien pour activer le compte.
4. Se connecter avec email + mot de passe.

Mot de passe oublié ? Utiliser **Mot de passe oublié** sur la page de connexion : un lien de réinitialisation est envoyé par email.



Figure 11.1 — Le formulaire d'inscription. Champs labellisés, contrainte de mot de passe affichée, message d'erreur explicite.

3. Créer et gérer un Nook

- **Créer** : bouton **Nouveau Nook** → saisir un nom → valider. Le Nook est créé avec une carte par défaut ; vous en êtes le propriétaire.

- **Inviter** : dans les réglages du Nook → **Inviter** → copier le lien et le partager. Toute personne ouvrant le lien rejoint le Nook.
- **Rôles & permissions** : attribuer des rôles aux membres (gérer les salons, gérer les membres...).
- **Modérer** : dans l'onglet **Membres**, un modérateur peut **exclure** (kick) ou **bannir** un membre. Les bannis ne peuvent plus revenir, même via un lien, jusqu'à leur **débannissement**.



Figure 11.2 — L'onglet Membres. Liste des membres avec leurs rôles, actions d'exclusion et de bannissement, onglet des bannis.

4. Discuter

- **Salons de texte** : sélectionner un salon dans la barre latérale et écrire un message (Entrée pour envoyer).
- **Éditer / supprimer** : survoler son propre message pour l'éditer ou le supprimer. Un message édité est marqué comme tel.
- **Messages privés (DM)** : ouvrir la messagerie, rechercher un utilisateur par son handle, et lui écrire directement.
- Le chat est **en temps réel** : les messages apparaissent instantanément chez tous les membres.

5. Le monde 2D

- **Se déplacer** : touches **ZQSD** ou **flèches directionnelles**. Le personnage s'anime selon la direction.
- **Voir les autres** : les membres présents apparaissent et bougent en temps réel.
- **Voix de proximité** : en entrant dans une **zone vocale** ou en s'approchant d'autres joueurs, la conversation audio s'active automatiquement ; elle se coupe lorsqu'on s'éloigne.
- **Personnaliser son avatar** : choisir l'apparence du personnage.

6. Éditer la carte (administrateurs)

Un administrateur du Nook peut ouvrir l'**éditeur de map** (3 phases) :

1. **Sols** — peindre les tuiles de sol case par case.
2. **Pièces** — tracer une pièce (murs + porte) ; créer une pièce crée automatiquement un salon de discussion associé.
3. **Décor** — glisser-déposer des objets (rotation, profondeur).

Plusieurs administrateurs peuvent éditer **en même temps** : les modifications fusionnent sans conflit et sont sauvegardées automatiquement.

7. Préférences

- **Thème** : basculer entre clair, sombre et pixel-quest (bouton en haut à droite). Le choix est mémorisé.
- **Langue** : basculer **FR / EN** ; l'interface est traduite intégralement.
- **Accessibilité** : navigation clavier avec focus visible et lien « Aller au contenu principal » ; respect de la préférence système « réduire les animations ».

8. Gérer ses données (RGPD)

Dans les réglages du compte :

- **Exporter** ses données personnelles.
- **Supprimer** son compte. Si vous possédez des Nooks, vous pouvez en **transférer la propriété** à un autre membre avant suppression ; sinon le Nook est supprimé. La suppression efface le compte et toutes ses sessions.

9. Aide rapide

JE VEUX...	où
Créer un espace	Bouton « Nouveau Nook »
Inviter quelqu'un	Réglages du Nook → Inviter
Écrire un message	Barre latérale → salon → zone de saisie
Parler en vocal	Entrer dans une zone vocale du monde
Changer la carte	Éditeur de map (admin)
Changer la langue/thème	Contrôles en haut à droite
Supprimer mon compte	Réglages du compte

Compétence C2.4.1

Livrable RNCP **C2.4.1** — Documentation technique d'exploitation (manuel de mise à jour). Décrit comment faire évoluer l'application en production en toute sécurité.

1. Principe

Les mises à jour sont **continues et automatisées**. Le cycle de vie d'une évolution est piloté par Git + GitHub Actions ; l'exploitant n'a normalement aucune commande à lancer sur le serveur.

```
Ticket → branche fix|feat/... → PR → CI verte → merge dev → déploiement staging  
→ validation recette → merge dev→main → déploiement production
```

2. Mettre à jour l'application (nominal)

1. **Développer** sur une branche dédiée (`feat/...` ou `fix/...`), commits en **Conventional Commits**.
2. **Ouvrir une PR** vers `dev` . La **CI** (lint, typecheck, tests, build) doit être verte pour fusionner.
3. **Fusion sur** `dev` → déploiement automatique sur **staging** (dev.nookapp.eu) + healthcheck.
4. **Valider** sur staging avec le « Cahier de recettes ».
5. **Promouvoir** en production : PR `dev` → `main` , fusion → déploiement automatique sur **nookapp.eu**.

À chaque déploiement, l'image est taguée par le **SHA du commit** (`<branche>-<sha>`) : une version en production est toujours reliée à un point précis de l'historique.

3. Migrations de base de données

Les évolutions de schéma sont **versionnées** avec Drizzle :

1. Modifier le schéma dans `packages/db/src/schema/` .
2. Générer la migration : `pnpm --filter @nookapp/db generate` → fichier SQL dans `packages/db/drizzle/` .
3. Committer la migration avec le code.

En production, les migrations s'appliquent **automatiquement au démarrage** du conteneur `api` (`docker-entrypoint.sh`), **avant** le lancement du serveur. Aucune commande manuelle.

Recommandation : effectuer une **sauvegarde** (voir §5) avant tout déploiement comportant une migration destructive.

4. Numérotation des versions

- Le versionnage suit **SemVer** et est automatisé par **release-please** : chaque push sur `main` met à jour une release PR (calcul de la version depuis les commits, mise à jour de `CHANGELOG.md`).
- Au merge de la release PR : tag `vX.Y.Z` + GitHub Release.
- `feat` → version mineure, `fix` → patch, changement cassant (`!`) → majeure.

Consulter `CHANGELOG.md` et « Historique et gestion des versions » pour le journal complet.

5. Sauvegarde avant mise à jour

Le service `db-backup` réalise des sauvegardes planifiées, mais une sauvegarde manuelle est recommandée avant une mise à jour sensible :

```
cd /opt/<stack>
docker compose -p <stack> exec db-backup /usr/local/bin/db-backup.sh
```

Les dumps sont conservés dans le volume `db-backups` selon la rétention configurée (`BACKUP_RETENTION_DAYS`).

6. Mettre à jour les dépendances

- Dependabot** ouvre automatiquement des PR de mise à jour (npm hebdomadaire, GitHub Actions mensuel), groupées par type (mineures/patch).
- Chaque PR passe par la **CI** avant fusion : une dépendance qui casserait le build ou les tests est bloquée.
- Traiter ces PR régulièrement pour rester à jour côté sécurité (cf. A06 dans « Sécurité — couverture OWASP Top 10 »).

7. Vérifications après mise à jour

- Healthcheck : `GET https://<domaine>/api/v1/health` → `200`, `db: true`.
- État des conteneurs : `docker compose -p <stack> ps` (tous `Up`).
- Contrôle fonctionnel rapide : connexion, envoi d'un message, entrée dans le monde.
- En cas d'anomalie : **rollback** immédiat en redéployant le tag immuable précédent (voir « Manuel de déploiement » §9), puis traitement via le « Plan de correction des bogues ».

8. Procédure d'urgence (rollback)

```
cd /opt/<stack>
export IMAGE_TAG=<branche>-<ancien-sha> # version stable connue
docker compose -p <stack> -f docker-compose.app.yml --env-file .env up -d
```

Le retour arrière n'affecte pas les volumes de données. Si une migration destructive a été appliquée, restaurer la base depuis la dernière sauvegarde (`scripts/db-restore.sh`).